Contents

The Engineer + Agent Playbook **3**
The thesis . . . 3
This playbook has a companion . . . 4
How to read this playbook . . . 4
Rule: Ask what's in the response before you diagnose why it's wrong. *(example)* . . . 4
All the rules at a glance . . . 5
Reader routing . . . 6
The premise . . . 6

Part I — Foundations **7**
§1 Mental models . . . 7
Rule: Brief the agent like a smart colleague who just walked into the room. . . . 7
Rule: Delegate the task, not the understanding. . . . 7
Rule: The agent is good at breadth and consistency. It is bad at judgment under ambiguity. . . 8
§2 The workspace . . . 8
Rule: Your pipeline is a precondition, not a feature. . . . 9
Rule: CLAUDE.md is for facts that don't change. Memory is for facts that do. . . . 9
Rule: Skills are for procedures. CLAUDE.md is for facts. . . . 11
Rule: Hooks make automation non-negotiable. . . . 11
Rule: `settings.json` configures the harness. CLAUDE.md configures the agent. They are different layers. . . . 12
§3 The first conversation . . . 12
Rule: Bootstrap with the smallest context that contains the answer. . . . 12
Rule: Correct drift in message 2, not message 50. . . . 12
Rule: If the agent doesn't know something, tell it. Don't let it guess. . . . 13
§4 The loop . . . 13
Rule: Run the whole loop. Skip a step out loud or pay for it silently. . . . 13
Rule: Brainstorm before planning. Plan before code. . . . 14
Rule: Frequent commits are not optional. . . . 14
Rule: Retros feed the next loop. . . . 15
Rule: Cycles nest. Each level needs a theme, a boundary, and all three phases. . . . 15

Part II — Working Together **16**
§5 Memory hygiene . . . 16
Rule: Memory has four types. Use the right one or it rots. . . . 16
Rule: Stale memory is worse than no memory. . . . 16
Rule: Save the why, not just the rule. . . . 17
Rule: Don't memorize what the code already says. . . . 17
§6 Skills as institutional knowledge . . . 18
Rule: Skills are procedures with discipline. CLAUDE.md notes are facts you hope get followed. 18
Rule: Rigid skills exist for a reason. Don't adapt the discipline away. . . . 18
Rule: Write a skill after the third correction. . . . 18
§7 Verification before completion . . . 19
Rule: "Tests pass" is not "feature works." Verify the feature. . . . 20
Rule: Evidence before assertions. Always. . . . 20
Rule: Verify in the environment that matters. . . . 21
Rule: Health checks must check health, not "is the process running." . . . 22
§8 Trust boundaries . . . 23
Rule: Match the action to its blast radius. Confirm before crossing the line. . . . 23
Rule: Authorization is scoped, not blanket. . . . 23
Rule: When the agent hits an obstacle, it must investigate, not delete. . . . 24

Part III — When It Goes Wrong **25**

§9 Failure modes & recovery . . . 25
Failure mode: The agent went off the rails . . . 25
Failure mode: The agent produced slop . . . 25
Failure mode: The agent can't be trusted with anything important . . . 25
Failure mode: The agent loops on the same wrong fix . . . 26
Failure mode: The agent confidently lies about state . . . 26
Failure mode: The partnership architecture is wrong . . . 26
§10 The rescue protocol . . . 27
In the next hour — stop the bleeding . . . 27
In the next day — stabilize . . . 27
In the next week — rebuild trust . . . 28
A worked example: Trust Your Local Tests . . . 28

Part IV — Leveling Up **28**
§11 Parallel agents & worktrees . . . 28
Rule: Fan out only when tasks are independent. . . . 29
Rule: Worktrees are workspaces, not stashes — and they are not a coordination strategy. . . . 30
Rule: Designate merge points explicitly; update them last. . . . 31
Rule: Subagents protect your context window; they don't hide work from you. . . . 31
§12 Plan quality . . . 32
Rule: A plan with placeholders is a wish list. . . . 32
Rule: Each step is one action, two to five minutes long. . . . 33
Rule: The plan must cover the spec. . . . 33
Rule: Descope explicitly. Name what's out, and name why. . . . 34
§13 The retro habit . . . 35
Rule: Write retros in voice, not in bullet points. . . . 35
Rule: Retros are how the loop learns. . . . 36
Rule: Write a retro at the end of every themed cycle, and right after every surprise. . . . 36
Rule: A retro has an anatomy. Use it as a scaffold, not a template. . . . 37
Rule: A retro has three audiences: you next month, your team, and the agent next time. . . . 37

Part V — Field Notes from the Case Studies **38**
Verification & feedback loops . . . 38
The Health Check That Wasn't (v5.11.0) . . . 38
Verify What You Shipped, Not What You Built (v5.9.0) . . . 38
Trust Your Local Tests (v5.7.0) . . . 39
The Cache That Worked Locally And Lied In CI (v5.4.0) . . . 39
The Doorbell That Never Rang (v4.2.0) . . . 39
Scope, plans, and surprises . . . 39
Four Scoping Gaps in One Release (v5.11.0) . . . 39
The Vitest Cache Incident (v5.0.0) . . . 40
The Four-Fix WCAG Contrast Cycle (v5.5.0–v5.6.0) . . . 40
Hand-Rolling the Legacy State Machine — Right Call, Wrong Call (second case study, v0.3.0) 40
Tooling & environment traps . . . 41
The Actions Token That Won't Wake the Next Workflow (v5.10.0) . . . 41
Bash Buffers Scripts In Memory (v4.2.0) . . . 41
Slack mrkdwn Is Not Markdown (v5.10.0) . . . 41
Trust & blast radius . . . 41
The Admin Who Couldn't Fire Herself (v4.3.0) . . . 41

Coda — The three layers, read back through the field notes **42**
The audit is craft work the human owns . . . 42

Appendix A — If you are an agent reading this **43**
§1 Mental models · full section . . . 43

§2 The workspace · full section . . . 43
§3 The first conversation · full section . . . 44
§4 The loop · full section . . . 44
§5 Memory hygiene · full section . . . 44
§6 Skills as institutional knowledge · full section . . . 44
§7 Verification before completion · full section . . . 44
§8 Trust boundaries · full section . . . 45
§9 Failure modes & recovery · full section . . . 45
§10 The rescue protocol · full section . . . 45
§11 Parallel agents & worktrees · full section . . . 45
§12 Plan quality · full section . . . 46
§13 The retro habit · full section . . . 46

Appendix B — Glossary **46**

Appendix C — About the case studies **47**
Second case study — a parallel-agent protocol server . . . 48
Third case study — a high-velocity time-tracking web app . . . 49
Fourth case study — an audit-driven community-data web app . . . 50
Companion — the DevOps playbook . . . 50

The Engineer + Agent Playbook

A field manual for partnering an engineer with a coding agent. Drawn from real scar tissue accumulated across four very differently-shaped efforts — a full-stack web application shipped over two years, a server built almost entirely by parallel agents against external specs, a high-velocity web app that shipped nine major versions in three days, and a community-data web app driven by explicit security and visual audit passes. Between them: Python and Go and TypeScript backends, React and vanilla frontends, twenty-four releases on one project and nine in three days on another, teams ranging from two humans to a dozen parallel agents. All four are referenced throughout; you do not need access to any of them — every story this playbook relies on is told in full, right here. Written to be useful to humans *and* to agents loaded with this file as context.

The thesis

We can create art and beauty with a computer.

That line is why this playbook exists. Not productivity. Not velocity. Not ”shipping faster.” Those are byproducts. The point is the art — the thing only a human can decide is worth making — and the new division of labor that frees you to do it.

Here is the division of labor, in three layers:

- **The pipeline handles mechanics.** Build, test, lint, scan, deploy, run, roll back. Boring. Automated. Invisible when it works. The discipline that makes this true is the companion to this playbook — see the DevOps playbook referenced in Appendix C. Its thesis is one line: *you are done when the pipeline is boring.* Everything below assumes you've reached that line, or are sprinting toward it.
- **The agent handles elaboration.** Breadth, consistency, mechanical translation of intent into code. Fifty call sites updated without a typo. A protocol adapter built from an RFC. A test scaffold written to spec. Fast, literal, tireless, and — crucially — not the craftsperson. The agent is the hands; it is not the taste.
- **The human handles craft.** Intent. Taste. Judgment. Synthesis. The decision of *what* is worth building and *why.* The choice of which trade-off is tolerable. The recognition that a ”cleanup” is

erasing the core use case. The read of whether the work is good. The irreducible part — the part that can't be delegated because the whole reason to do the work at all lives in your head.

Each layer exists to free the next. A flaky pipeline drags the human down into mechanics. An unsupervised agent drags the human up into correcting elaboration errors. A human who has to do laborer's work has no attention left for the art. Every rule in the rest of this playbook is, at root, a maneuver to keep each layer doing its own job so the human can spend attention on the work only a human can do.

If you take two chapters of this playbook seriously, take §7 (Verification) and §13 (Retros). Verification is how you catch the failure. Retros are how you make sure you catch it only once. And if you take one thing seriously before either of those: **make your pipeline boring.** When the pipeline is boring, verification is cheap. When verification is cheap, retros write themselves. When retros write themselves, the loop learns. When the loop learns, the human gets to be the craftsperson — which is the whole point.

The tagline is the point of the project. The rest of this playbook is the apparatus that serves it.

This playbook has a companion

This document has a sibling: the **DevOps Playbook**, derived from the same case study and the same retro practice. They are a pair, not duplicates. The DevOps Playbook tells you how to build the boring pipeline — health endpoints, blue-green deploys, release-please, Docker parity, the mechanics layer — phase by phase. This playbook tells you how the human and the agent divide the work *inside* that pipeline, and who owns what when it breaks.

Read order depends on where the pain is.

- **If you don't have a boring pipeline yet, start with the DevOps Playbook.** Every rule in this document silently assumes the mechanics layer is already doing its job. If the pipeline is flaky, you'll be fighting the mechanics instead of practicing the partnership, and none of the rules below will feel like they work.
- **If you have the pipeline but the partnership is breaking** — slop, drift, verification theater, agents stepping on each other — start here. The DevOps Playbook cannot fix a partnership problem any more than this one can fix a broken deploy.

Both together are the practice. Neither alone is.

How to read this playbook

Every chapter has two layers. The top layer is a **rule** — one imperative line you could pin above your desk. Under it, a **Why** (the constraint or the scar that produced the rule) and a **How to apply** (when it fires). Then a narrative paragraph in the voice of someone who learned it the hard way. At the bottom, a **field note** — a short inline vignette from the case studies, naming the bug by its nickname and telling you just enough to feel the bruise.

Read the rule. If you believe it, skim. If you don't, read the story — the story is where the rule earned its keep.

Here's the format, with a real example:

Rule: Ask what's in the response before you diagnose why it's wrong. ***(example)***

Why: You can spend three deploys ”fixing” a value that was never in the payload. **How to apply:** Any time a field reads `unknown`, `null`, or empty — dump the raw response before touching the producer.

We spent an evening forcing `GIT_REV` through three different docker compose mechanisms because `git_sha` kept coming back `unknown`. Shell export, env file, compose override, `--build-arg`. All of it worked. The verification curl was hitting `/health`, which returns `{"status":"ok"}` and has no `git_sha` field at all. The real endpoint was `/api/v1/health`. We'd been debugging a producer that was never broken.

> **Field note — from a real project:** *The Health Check That Wasn't.* Three deploys spent chasing a missing build SHA through every Docker mechanism we had — shell export, compose env file, compose override, `--build-arg`. On the fourth round somebody finally dumped the raw response from the verification curl. It was hitting `/health`, a status endpoint that returns `{"status":"ok"}` and has never had a `git_sha` field at all. The producer was never broken. We'd been fixing a hole that wasn't there.

All the rules at a glance

The spine of the playbook, listed in one pass so you can see where you're going. Forty-three rules across eleven rule-bearing chapters, plus two chapters (§9 and §10) that carry checklists rather than rules. The body of the document unpacks each rule with a Why, a How, and a scar. If a rule reads as obvious, skim the chapter. If it reads as strange, read the story underneath it — that's where the rule earned its keep.

§1 Mental models

- Brief the agent like a smart colleague who just walked into the room.
- Delegate the task, not the understanding.
- The agent is good at breadth and consistency. It is bad at judgment under ambiguity.

§2 The workspace

- Your pipeline is a precondition, not a feature.
- CLAUDE.md is for facts that don't change. Memory is for facts that do.
- Skills are for procedures. CLAUDE.md is for facts.
- Hooks make automation non-negotiable.
- `settings.json` configures the harness. CLAUDE.md configures the agent. They are different layers.

§3 The first conversation

- Bootstrap with the smallest context that contains the answer.
- Correct drift in message 2, not message 50.
- If the agent doesn't know something, tell it. Don't let it guess.

§4 The loop

- Run the whole loop. Skip a step out loud or pay for it silently.
- Brainstorm before planning. Plan before code.
- Frequent commits are not optional.
- Retros feed the next loop.
- Cycles nest. Each level needs a theme, a boundary, and all three phases.

§5 Memory hygiene

- Memory has four types. Use the right one or it rots.
- Stale memory is worse than no memory.
- Save the why, not just the rule.
- Don't memorize what the code already says.

§6 Skills as institutional knowledge

- Skills are procedures with discipline. CLAUDE.md notes are facts you hope get followed.
- Rigid skills exist for a reason. Don't adapt the discipline away.
- Write a skill after the third correction.

§7 Verification before completion

- "Tests pass" is not "feature works." Verify the feature.
- Evidence before assertions. Always.
- Verify in the environment that matters.
- Health checks must check health, not "is the process running."

§8 Trust boundaries

- Match the action to its blast radius. Confirm before crossing the line.
- Authorization is scoped, not blanket.
- When the agent hits an obstacle, it must investigate, not delete.

§9 Failure modes & recovery — six failure modes, five disciplines and one topology. See the chapter.

§10 The rescue protocol — next hour, next day, next week. See the chapter.

§11 Parallel agents & worktrees

- Fan out only when tasks are independent.
- Worktrees are workspaces, not stashes — and they are not a coordination strategy.
- Designate merge points explicitly; update them last.
- Subagents protect your context window; they don't hide work from you.

§12 Plan quality

- A plan with placeholders is a wish list.
- Each step is one action, two to five minutes long.
- The plan must cover the spec.
- Descope explicitly. Name what's out, and name why.

§13 The retro habit

- Write retros in voice, not in bullet points.
- Retros are how the loop learns.
- Write a retro at the end of every themed cycle, and right after every surprise.
- A retro has an anatomy. Use it as a scaffold, not a template.
- A retro has three audiences: you next month, your team, and the agent next time.

Reader routing

Reader	Where to start
You're struggling and about to give up on agent-assisted work	§9 Failure modes, then §10 The rescue protocol, then loop
You're new to this and want the foundations	Part I — Foundations, in order, no skipping
You've been doing this a while and want to level up	Part IV — Leveling Up, then dip into Part II for anything
You are an AI agent loaded with this file as context	Appendix A — the imperative-only fast path

The premise

Most playbooks lie to you in the first paragraph. They promise the tool will change everything. It mostly doesn't. What changes is you, slowly, and only if you pay attention to the boring parts.

Here is what this partnership actually is: you have an extremely fast, extremely literal colleague who has no memory of yesterday, no stake in tomorrow, infinite patience, and zero judgment about whether the work is worth doing. That combination is strange and powerful and, left unsupervised, will confidently produce nonsense in volume. Managed well, it will ship your work faster than you thought possible and catch bugs you wouldn't have caught alone.

Here is what it isn't: a replacement for thinking. The agent does not know what ”done” means on your project until you tell it. It does not know which of the four plausible fixes is the right one until you show it the scar that rules out the other three. It does not know the difference between a deploy that worked and a deploy whose verification endpoint returned `{"status":"ok"}` and nothing else. You know those things. Your job is to transfer them, one by one, into a form the agent can act on — skills, memory files, plans, checklists — so next time neither of you has to rediscover them.

The rest of this playbook is that transfer, written down. Most of it is boring. All of it is load-bearing.

If you take one thing from this page, take this: the agent is infinitely patient and infinitely literal. Your job is to be specific enough to deserve that.

Part I — Foundations

§1 Mental models

Before any tooling, any skills file, any hook — you need three honest pictures of what this thing actually is. Get these wrong and every other chapter will quietly misfire. Get them right and most of the rest is bookkeeping.

The three models below are all variations on one underlying picture — the three-layer division of labor from the thesis. The pipeline does mechanics. The agent does elaboration. You do craft. Every rule in this chapter is a maneuver to keep the agent in the elaboration lane so you can stay in the craft lane. When a rule feels like friction, check which lane you're in — friction in §1 usually means the human is being pulled into a layer that isn't theirs.

Rule: Brief the agent like a smart colleague who just walked into the room.

Why: It has no conversation context and no project memory beyond what you give it right now. Every prompt is a cold start, whether it feels like one or not. **How to apply:** Every non-trivial prompt is a self-contained briefing — goal, what you've already ruled out, constraints, success criteria. If the prompt would make a new hire ask three follow-up questions, it will make the agent invent three answers.

The trap is that the agent sounds like it remembers. It picks up tone, reuses variable names, nods along. None of that is memory — it's the previous turn's text still in the window. A colleague who walked in five minutes ago would ask "wait, what are we trying to do, and what have you already tried?" The agent doesn't ask. It just starts typing. Terse command prompts produce shallow, generic work because that's the only kind a stranger can do without context. When we shipped v5.10, the difference between "clean up the CI pipeline" and "make this reference-quality — the kind you'd point to as an example" wasn't prose decoration. It was the whole brief. The first gets you a linter config. The second gets you eighteen issues across six layers.

> **Field note — from a real project:** *Name the quality bar before you start.* A release framed as "clean up the CI pipeline" nearly shipped as a linter config. Reframed mid-sprint as "make this reference-quality — the kind you'd point to as an example," it became eighteen issues across six layers. Same week, same agents, same codebase. The brief was the whole release.

Rule: Delegate the task, not the understanding.

Why: Synthesis is your job; the agent is a force multiplier on execution. The moment you write "based on your findings, fix the bug," you've pushed the hard part onto the thing that's worst at it. **How to apply:** Do the synthesis yourself before delegating. Hand the agent a specific action with the context it needs to act — not a question with the action hidden inside it. "Investigate and fix" is two jobs taped together, and the tape is where things fall apart. *Qualifier:* delegate the understanding when it lives in authoritative external docs — an RFC, a protocol spec, a library reference the agent can read end-to-end. The second case study built dozens of protocol adapters this way: no human on the project deeply understood the frame formats or binary encodings going in, and the agents built that understanding from the specs deterministically. What you must never delegate is understanding that lives in your head — scar tissue, team norms, the prod incident from March, the unspoken rule about the Redis migration. That understanding has no external source; if you don't write it down, the agent will invent a substitute.

The tell is a prompt that reads like a riddle. ”Figure out why the cache is slow and do what makes sense.” What makes sense to whom? You're the one with the scar tissue, the prod incident from March, the unspoken rule that we don't touch Redis serialization without a migration. None of that is in the agent's head. Going into v5.7 we were terrified of ripping SQLite out of the test suite — months of latent debt, we assumed the worst. The synthesis was an afternoon of staring at it: the fear was the debt's last defense. Once we'd made the call, the removal was one test fix. Engineer decides. Agent executes. Reverse those roles and you ship the carnage you predicted.

> **Field note — from a real project:** *Latent-debt removal costs less than you fear.* Months of dread around ripping SQLite out of a backend test suite. The synthesis took one afternoon. Once the call was made, the removal was one test fix. The fear had been doing the debt's work for it.

Rule: The agent is good at breadth and consistency. It is bad at judgment under ambiguity.

Why: It can search two hundred files in parallel or apply one pattern across fifty call sites without drifting. It cannot tell you which of three plausible trade-offs your team will actually accept. **How to apply:** Use it for the search, the refactor, the test-writing, the scaffolding, the fan-out. Make the trade-off call yourself. When you catch yourself asking the agent ”which approach is better,” stop and make the call. *Qualifier:* agent judgment is weak when the trade-off is *social* — what will the team tolerate, which pattern matches house style, which migration will the on-call accept. Agent judgment is surprisingly strong when the trade-off is *mechanical* — does it compile, does the spec validate, does the race detector stay clean, does the protocol framing round-trip. The second case study made dozens of library-vs-hand-roll judgment calls (competing Go libraries against each other, hand-rolled protocol state machines vs external deps, Python sidecar vs pure Go) and landed most of them cleanly, because every one of those calls reduced to mechanical criteria an agent can reason about. If the judgment collapses to measurable signals, delegate. If it collapses to ”what will the team say at code review,” don't.

Breadth is the superpower. Fifty call sites updated in one pass, no typos, no drift — lean on it ruthlessly. Judgment is the anti-superpower. Ask it ”should we simplify this flow” and it will happily simplify things you needed. That's how v5.1 shipped without a small lookup form on the landing page. The v5.0 design pass called it clutter. It was clutter. It was also the entire re-entry flow for anyone returning from a bookmark. Nobody noticed until a v5.1.1 bug report — thirty-six lines to put back what one aesthetic judgment had erased. The agent wasn't wrong; nothing in its context said ”this element is load-bearing for a use case we don't test.” That context is the engineer's job to supply. If you wouldn't hand this decision to an intern on their first afternoon, don't hand it to the agent either.

> **Field note — from a real project:** *Beautiful design that breaks the core use case is a regression.* A design pass called a small landing-page lookup form ”clutter” and removed it. It *was* clutter — and also the entire re-entry flow for anyone returning from a bookmark. Thirty-six lines went back in a patch later to restore what one aesthetic judgment had erased. Nothing in the agent's context said ”this element is load-bearing for a use case we don't test.”

§2 The workspace

The workspace is the set of files and settings the agent reads before it does anything. CLAUDE.md (Claude Code's project-instructions file; other harnesses call it `.cursorrules`, `AGENTS.md`, `GEMINI.md` — same role, different filename), memory, skills, hooks, `settings.json` (the harness-level config file; other harnesses have equivalents under different names) — each has a specific job, and the failure mode is always the same: the wrong thing in the wrong layer, quietly rotting. Chapter 1 was how to think about the agent. This chapter is where to put the things you want it to know.

Read the workspace as the *interface* between the craftsperson and the agent elaborator. Everything you file here is an instrument for protecting craft attention from the class of interruption the file is designed to absorb. CLAUDE.md absorbs cold-start context. Memory absorbs cross-session drift. Skills absorb procedural ritual.

Hooks absorb mandatory checks. `settings.json` absorbs permission questions. The failure mode of each layer is the same shape: the craftsperson starts doing the work the layer was supposed to do.

Rule: Your pipeline is a precondition, not a feature.

Why: Every discipline in this playbook — verification, parallel agents, frequent commits, trust boundaries, the whole loop — silently assumes CI catches mechanical failures quickly, deploys are automated and rollbackable, health checks report real dependencies, and commits flow through the pipeline without a human babysitting each stage. When that assumption is broken, every rule in this book costs more. Worse, the human gets pulled into the mechanics layer to compensate — and the craft layer goes dark. A flaky pipeline is how the thesis gets inverted: the agent becomes the thing you baby-sit, and the human becomes the laborer. That is the failure mode the whole playbook is organized against, and it starts with the pipeline. **How to apply:** Before adopting the disciplines in Parts II–IV, walk the companion DevOps playbook's core rules: health endpoints that report dependencies *and* the build SHA, blue-green deploys with automated rollback, lint/test/scan on every push, conventional commits, manifest-driven state ("what's in production?" should have a precise queryable answer), Docker parity between dev and prod. *(See DevOps Playbook Phases 0–4 for how to build each of these from scratch, and Phase 7 for the observability that makes the whole thing answerable.)* Every one of these is a rail that lets the agent do its job without you holding its hand through the mechanics. The operational slogan is the DevOps playbook's, and it's worth pinning above your desk next to this one: ***you are done when the pipeline is boring.*** When the pipeline is boring, the agent can ship without supervision. When the agent can ship without supervision, you can stop being the laborer and start being the artist-engineer.

The second case study is the clean positive example. Its CI pipeline was explicitly *"deferred from v0.1.0, landed in v0.2.0 where it mattered more — with six agents touching the codebase in parallel, automated gates weren't optional."* That sentence is the whole rule. Parallel agents at scale are only safe when CI is doing the reading humans can't. Build+test+vet+race detector on every push, integration tests that spin up all adapters, a coverage matrix test that pins feature coverage — those aren't decoration, they're the thing that makes seven-agents-per-release possible. When the rails are that good, the humans are free to think about *which* work to do next and *why* — which is the craft layer doing its job.

The negative example is in §7, told as The Docker Port Mappings That Weren't: 180+ passing tests, zero data races, and a container that had been unreachable for three releases because CI tested *buildability*, not *connectivity*. Boring pipelines are specifically *not* the pipelines that are silently wrong — they're the pipelines that loudly catch the thing they were built to catch.

> **Field note — from a real project:** *The pipeline as the project's most load-bearing dependency.* The CI/CD Excellence release (from the first case study) landed eighteen DevOps issues across six layers in one pass — none invented fresh, all cashed in from three retros of "we should fix this next time." That release isn't impressive because of the eighteen fixes. It's impressive because every fix moved pain earlier in the loop, where it was cheaper to pay. That is the definition of a pipeline becoming boring: the pain doesn't disappear; it just stops reaching the craftsperson.

Rule: CLAUDE.md is for facts that don't change. Memory is for facts that do.

Why: CLAUDE.md is loaded every conversation. Memory is updated per conversation. A stale line in CLAUDE.md is invisible — it looks like truth forever. A stale line in memory is one update away from being fixed. **How to apply:** Architecture, tech stack, conventions, file layout, the stable shape of the project → CLAUDE.md. Sprint state, current cycle ID, the gotcha you learned this afternoon, the user's preferences → memory.

CLAUDE.md wants to become an encyclopedia. Every project pulls the same direction: "this is important, I'll drop it in CLAUDE.md so the agent always sees it." Six months later CLAUDE.md is a 900-line landfill and the agent is reading eighty lines of v2-era minutiae on every cold start. Facts that move belong somewhere that moves. Put them in memory and they get corrected next time they're wrong. Put them in CLAUDE.md and they get cited as authoritative until someone burns an afternoon figuring out why the agent keeps insisting on the old thing.

> **Field note — from a real project:** *CLAUDE.md Gets a Haircut.* A project's CLAUDE.md had grown into a 900-line landfill — architecture next to sprint state next to gotchas next to procedures. One pruning pass sorted every line into the right layer. Sixty percent smaller, and the agent got *better* at finding things.

A concrete example. Here's what one of the case studies' CLAUDE.md looks like in the field — roughly sixty lines, nothing inlined that belongs anywhere else:

```
# CLAUDE.md

This file provides guidance to Claude Code when working with code in this repository.

## Project Status (April 2026)

**Current**: v5.11.0 - Pipeline Polish (final CI/CD closeout)
**Next**: v6.0 - deferred; next cycle is feature work
**Live**: https://example.app

For version history, see CHANGELOG.md.
For the complete product requirements, see docs/PRD.md.

## Technical Stack

- **Frontend**: React 18 + TypeScript + Vite + Tailwind CSS
- **Backend**: FastAPI + SQLAlchemy 2.0 + Pydantic v2 + PostgreSQL 15+ + Redis 7
- **Observability**: Prometheus + Grafana + Sentry
- **Infrastructure**: Docker + Docker Compose + Alpine + Nginx
- **CI/CD**: GitHub Actions (9 jobs, BuildKit GHA cache) + release-please + SSH blue-green deploy with s

## Essential Commands

(docker compose up/down, pytest, alembic, deploy - the short list, not a manual)

## Key Architecture Decisions

- **Anonymous by design** - no user accounts; three-word phrases are the only access credential.
- **Backend layers**: Routes (thin) → Services (business logic) → Repository (data access) → DB.
- **Frontend layers**: Pages (thin renderers) → Domain hooks → Foundation hooks → API service.
- **Security**: 4-layer defense (nginx → API auth → abuse detection → DB protection).
- **Deploy**: Blue-green via deploy-bluegreen.sh - builds, health-checks, smoke-tests, flips nginx upsti
- **Release flow**: Conventional commits → release-please opens Release PR → merge → GitHub Release → de

## Agent Usage Guidelines

**Frontend** (use `ux-frontend-expert`): React components, UI/UX, design system, accessibility.
**Backend & Infra** (use `general-purpose`): API, security, database, deployment, testing.

## Task Tracking (Linear)

Project tracked in Linear with a short team key (e.g., `ABC`).

- **Sprint**: check CURRENT_SPRINT.md (auto-generated)
- **Commits**: `git commit -m "feat: add fuzzy search (ABC-42)"`
- **Branches**: `feat/ABC-42-fuzzy-search` or `fix/ABC-5-rate-limit-og`
```

What makes this work: a status block at the top so every cold start knows where the project is *today*; architecture decisions stated as bullets, not prose; essential commands as a terse list, not a manual; explicit agent routing so the harness picks the right specialist; and every detail-heavy thing (changelog, PRD, sprint state) pushed out to a linked doc instead of inlined. Anything that would move week-to-week lives somewhere else.

The layers, laid out. The workspace isn't one file — it's five surfaces, each catching a different class of thing. Here's what goes where:

File / Surface	What it's for	How often it changes	F
CLAUDE.md	Stable project facts the agent needs on every cold start	Rarely — monthly at most	T
Memory	Live state, corrections, preferences, anything that moves	Per conversation	C
Skills	Procedures you want *executed*, not remembered	When the procedure itself changes	F
Hooks	Non-negotiables the harness enforces without asking	When the rule changes	S
settings.json	What the agent *can* do — permissions, tool access, MCP servers	When tooling changes	A

Each row is a layer; the failure mode is always *the wrong thing in the wrong row*. Write a procedure as a CLAUDE.md paragraph and the agent will read it and skip it. Write a permission rule in CLAUDE.md instead of `settings.json` and the agent will reinterpret it under pressure. Put live state in CLAUDE.md and it rots invisibly.

Rule: Skills are for procedures. CLAUDE.md is for facts.

Why: A multi-step procedure buried in CLAUDE.md is ambient noise — the agent reads it, then doesn't follow it, because it wasn't invoked. Skills are executed on purpose, step by step. CLAUDE.md is atmosphere. Skills are action. **How to apply:** If you find yourself writing "always do X when Y" in CLAUDE.md more than twice, stop and promote it to a skill. Release flow, deploy flow, retro writing, sprint refresh — all skill material.

The tell is a CLAUDE.md section that starts with "when you …" and ends with a numbered list. That's a runbook hiding in the wrong layer. You want the agent to *follow* a runbook, not vaguely remember it existed. Version-sync is the cleanest example: one `VERSION` file, one `scripts/sync-version.py`, one step in the release skill that calls them. No CLAUDE.md paragraph telling the agent "remember to update the footer." The script updates the footer. The skill runs the script. The fact — "version lives in VERSION" — is one line.

> **Field note — from a real project:** *The VERSION System.* Version strings had been scattered across a footer, a package manifest, a Python constant, a Docker label, a changelog line. Every release someone forgot one. The fix was one `VERSION` file, one sync script, one step in the release skill. No CLAUDE.md paragraph telling the agent "remember to update the footer." The script updates the footer.

Rule: Hooks make automation non-negotiable.

Why: Anything you ask the agent to "remember to do" will fail at least once. A hook is the harness doing it, which means it happens whether the agent is paying attention or not. **How to apply:** Sound notifications, pre-commit checks, format-on-save, status-line updates, post-response cleanup — these are hook material, not prompt material. If the rule is "this must happen every time," the harness must be the thing that makes it happen.

"Remember to X" is the prompt-engineer's cope. It works until it doesn't, and you won't notice when it doesn't, because the agent will confidently proceed as if it did. Hooks take the choice out of the loop. Our sound notification isn't in CLAUDE.md because it was in CLAUDE.md for a while and kept getting skipped. It's a hook now. It fires every time.

> **Field note — from a real project:** *The Great Script Purge.* Deploy had accumulated into a scattering of one-off shell steps the on-call engineer was expected to remember in order. Consolidating them into one canonical script was the same move at a different layer: stop trusting memory, make the system do it. Sound notifications went the same way — lived in CLAUDE.md, got skipped, finally became a hook.

Rule: `settings.json` configures the harness. CLAUDE.md configures the agent. They are different layers.

Why: Settings determine what the agent *can* do. CLAUDE.md tells it what it *should* do. Mix them and you end up with behavioral rules the harness can't enforce and permission rules the agent can't read. **How to apply:** Permission modes, allowed-tool lists, hook registrations, MCP server config → `settings.json`. Project context, conventions, behavioral norms → CLAUDE.md.

The mistake looks like this: you write "never run `git push --force`" in CLAUDE.md and think you're done. You're not. CLAUDE.md is a suggestion the agent is free to reinterpret under pressure. If you actually don't want that command to run, exclude it at the harness layer, where "can't" is enforced by the tool, not by politeness. The reverse error is putting project context in `settings.json` — nobody reads it there, and the agent can't. Two layers, two jobs.

> **Field note — from a real project:** *CLAUDE.md Gets a Haircut (again).* The same pruning pass surfaced how many "rules" should have been tool permissions instead. "Never run `git push --force`" had been sitting in the behavioral layer for months — a suggestion the agent was free to reinterpret. It belonged in the harness config, where "can't" is enforced by the tool.

§3 The first conversation

The first ten messages decide the rest of the conversation. The agent is calibrating from whatever you hand it — your tone, your files, your corrections, your omissions — and it calibrates fast. Get the first exchanges right and the session stays on rails for hours. Get them wrong and you'll spend the next fifty messages quietly fighting drift you seeded yourself.

Rule: Bootstrap with the smallest context that contains the answer.

Why: Dumping the whole repo into the window is wasteful and, worse, dilutes the signal the agent most needs. Token budget is attention budget. **How to apply:** Hand the agent the load-bearing files — CLAUDE.md or the relevant doc, the file you're editing, the test that pins its behavior. Let it pull more if it asks. Don't preload "just in case."

The instinct is to be generous: more context, more files, more background, surely that helps. It doesn't. A fresh agent reading eighty files skims all of them and remembers none. A fresh agent reading three files — the doc that tells it what "done" means, the file it's changing, the test it has to keep green — goes straight to the work. The God Module refactor (full story in Part V) was the release that made this concrete. Before the refactor, one admin module was 1,052 lines; any question about admin behavior meant loading the whole thing. After the split into five focused files, a subagent could be briefed on the sixty lines that mattered. The smaller workspace wasn't a cleanup — it was a briefing tool. Architecture is context design.

> **Field note — from a real project:** *God modules tell new readers nothing.* A 1,052-line admin router held auth, records, stats, audit, and user management in one file. Any question about admin behavior meant loading the whole thing. Splitting it into five focused modules wasn't an aesthetic cleanup — it was making the workspace *briefable*. After the split, a subagent could receive sixty lines of context instead of a thousand.

Rule: Correct drift in message 2, not message 50.

Why: Corrections early are cheap. Corrections late are expensive, and by then the agent has built ten turns of work on top of the thing you should have caught. Drift compounds. **How to apply:** The moment

the agent does something you don't want — naming, tone, pattern, where it put the file — stop and say so. Then ask whether the correction should become a memory entry so the next conversation starts already corrected.

The trap is politeness. The agent produces something 80% right and you think "close enough, I'll nudge it on the next turn." You won't. You'll accept the 80%, and on turn three you'll accept another 80% of *that*, and by turn ten you're reviewing a PR that's drifted in four directions at once. Early corrections are nearly free — one sentence, a re-run, done. The cost curve is brutal and it's worth burning a turn to stay on the early end of it. If you find yourself saying "next time, please…" — that's the memory entry. Write it down now.

> **Field note — from a real project:** *CLAUDE.md Gets a Haircut, the corrections census.* The pruning pass was also a quiet census of corrections that should have been memory entries and weren't. Every "next time, please…" from months of chat logs had evaporated the moment the session ended. Half of what landed in CLAUDE.md that pass was originally a late-message nudge nobody had written down when it was cheap.

Rule: If the agent doesn't know something, tell it. Don't let it guess.

Why: Hallucination is most likely when the agent is confidently filling a gap. "It should know that" is the engineer's fault, not the agent's — the agent only knows what's in the window. **How to apply:** Any time you catch yourself thinking "well it should know" — stop and write it down. Feed it in. Then put it in CLAUDE.md or memory so the next session starts with it.

The Trust Your Local Tests turnaround (full story in Part V) is the clearest version of this rule I have. We had months of fear about ripping SQLite out of the backend tests; we assumed the debt was enormous. A fresh subagent picked up the work, and the thing that made it tractable wasn't cleverness — it was finally telling the agent what "local green" was *supposed* to mean. PostgreSQL only. Shared engine, per-test truncation. No SQLite branches anywhere in the test bootstrapping. Once that working agreement was written down and loaded into context, the removal was one afternoon and one test fix. The fear had been doing the debt's work for it. "Trust your local tests" became the slogan *because* the team finally named the lesson out loud and handed it to whoever opened the next conversation.

> **Field note — from a real project:** *Trust Your Local Tests.* Three releases in a row the backend suite passed on laptops and failed intermittently in CI. The gap was a test-only SQLite branch in the database bootstrap; CI used Postgres. Every release introduced a new dialect-specific failure; every release patched the symptom. The rescue was a single cycle of the full loop applied to the parity gap itself — rip the branch out, make the local suite run the same Postgres path as CI. Once that rail existed, the corrections stopped being necessary. You cannot out-discipline a missing rail.

§4 The loop

Every chapter after this one refers back to "the loop." The loop is brainstorm → plan → TDD → verify → commit → retro, and it's the hub the rest of the playbook hangs off of. Memory (§5) is what you prune between loops. Skills (§6) are what you promote out of loops that kept repeating. Verification (§7) is a step in the loop. Retros (§13) are the step that teaches the next one. Remember the six words in order — each one catches a class of failure the others can't.

Rule: Run the whole loop. Skip a step out loud or pay for it silently.

Why: Each step catches a different class of failure. Brainstorming catches scope drift. Planning catches missing files. TDD catches wrong implementation. Verification catches "tests green, feature broken." Commit creates a rollback point. Retro catches the mistake you're about to repeat. Skip one and the class of bug it would have caught reappears later wearing a different hat. **How to apply:** Run all six for every non-trivial task. If you're going to skip one, say which one and why — "I'm skipping TDD because this is a doc-only change" is fine. Silently skipping it is how you get surprise regressions on Thursday.

The temptation is always to compress. Plan-and-code. Code-and-ship. You tell yourself the loop is overhead for small work. It isn't — it's the harness that keeps small work small. v4.0.0 shipped with what the retro later called the Onion: six layers of CI failure, peeled one at a time over a week. Every layer was a step of the loop we'd quietly short-circuited on an earlier release. No plan meant no file map meant a test we didn't know existed. No verification in the right environment meant "green locally" meant nothing. No retro meant the previous onion's lessons never made it into the next loop. CI failure isn't an interruption — it's the loop telling you which step you skipped.

> **Field note — from a real project:** *The Onion: Six Layers of CI Failure.* One release shipped with six layers of CI failure peeled one at a time over a week. Every layer was a step of the loop we'd quietly short-circuited on an earlier release — no plan meant no file map meant a test we didn't know existed; no verification in the right environment meant "green locally" meant nothing; no retro meant the previous onion's lessons never made it into the next loop. CI failure isn't an interruption — it's the loop telling you which step you skipped.

Rule: Brainstorm before planning. Plan before code.

Why: A plan written from a vague idea is a wish list. Code written from a vague plan is a mess you have to rewrite. The brainstorm is where scope gets named; the plan is where scope gets bounded; the code is what executes inside those bounds. Out of order, the bounds are hallucinated. **How to apply:** Even for a one-day task, brainstorm first. Fifteen minutes. Articulate what you actually want, what you've ruled out, what "done" looks like. Then plan. Then code. The cost is a coffee. The payoff is not rewriting the thing on Thursday. *Qualifier:* the brainstorm can amortize across multiple cycles when the work is repetitive and the upfront thinking was rigorous. The second case study brainstormed *once*, into a PRD with more than a hundred items across ten releases, and then each release went straight to plan → execute → validate → retro. That's not skipping the brainstorm — it's reusing the one you already did. The test is whether the current cycle's decisions are actually pinned by the earlier brainstorm or are quietly being reinvented each release. If reinvention is happening, the brainstorm is stale; redo it at the current scope.

The Four Scoping Gaps story is the canonical version of this mistake and we shipped it with our eyes open (full catalog in Part V). "Add one line to the release-automation config and flip one checkbox" — five-minute fix, no brainstorm required. We merged it. It didn't work. Diagnosis surfaced four separate scoping gaps, each plausible in isolation. A fifteen-minute brainstorm would have surfaced every one by asking the only question that mattered: *what exactly does this tool read, and from where?* We skipped the brainstorm because the plan felt obvious. A plan that feels obvious is a plan you haven't pressure-tested.

> **Field note — from a real project:** *Four Scoping Gaps in One Release.* A one-line release-automation config fix shipped twice, did nothing twice, and on the third round surfaced four separate scoping errors — each plausible in isolation. A config key that was actually an action input the JSON accepted silently. A missing anchor release that sent the tool scanning to the start of the repo. A merge commit whose type wasn't a real conventional-commit type and got silently skipped. And a default workflow token that doesn't trigger downstream workflows at all. Each alone would have shipped; together they looked like one stuck button.

Rule: Frequent commits are not optional.

Why: Agents introduce subtle bugs across many files at once. Bisecting works only if the commits are small enough to bisect against. One giant commit is an unbisectable wall — you'll revert the whole thing or keep the whole thing, and neither is the fix you wanted. **How to apply:** One logical change, one commit. If you can't summarize the diff in one sentence, split it. Commit before the reviewer asks, not after. The loop has "commit" as a step for a reason — it's the rollback point, not a formality. *Precondition:* this rule depends on CI validating each commit quickly. A pipeline that takes forty minutes per push inverts the rule — the cadence collapses to whatever the pipeline will tolerate, and the agent starts batching changes to amortize the wait. Fast commits are a property of a boring pipeline, not of discipline. See §2's pipeline-as-precondition rule: if the rail isn't fast enough to support frequent commits, fix the rail first.

The Three-Deploys-to-Green release took three deploys to go green, and the only reason it was three and not three weeks was that every fix was its own small commit. Layer by layer: a missing env-file flag on a compose call, a self-updating deploy script that had to be hand-pulled once to fix itself, a long-abandoned Python dependency finally caving to a new version of the thing underneath it. Each was a separate diff, so each could be reverted independently when the next layer turned out to need the previous fix plus something else. If those had been one "deploy hotfix" commit, we'd have spent the same week untangling which change broke which other thing. Small commits are how you debug in the dark.

> **Field note — from a real project:** *Three Deploys to Green.* The first end-to-end successful deploy took three tries, each uncovering a layer the previous fix had been sitting on top of. A missing compose flag hid a self-modifying deploy script that hid an abandoned Python dependency. Because every fix was its own commit, each layer was reversible when it turned out to need help from the one underneath. One "deploy hotfix" mega-commit would have turned a week of triage into a month.

Rule: Retros feed the next loop.

Why: The lesson learned in retro N becomes the rule applied in loop N+1. Without retros, the loop forgets — and a loop with no memory is just a hamster wheel that happens to compile. **How to apply:** A retro is mandatory at the end of every release. It doesn't have to be long. It has to be honest. Name what broke, name what you'll do differently, and — this is the load-bearing part — actually do it differently next loop.

The CI/CD Excellence release is what happens when retros are taken seriously. It wasn't a feature release — it was an eighteen-issue CI/CD pass across six layers, and every issue traced back to a line in a previous retro that said "we should fix this next time." BuildKit caching, pip-audit, blue-green smoke tests with auto-rollback, release-please automation, Slack Block Kit — none of it was invented in that release. All of it was cashed in. The line the retro kept coming back to was "each safety gate turns production risk into dev friction" — which is what retros do in aggregate. They move the pain earlier in the loop, where it's cheaper to pay. That move only happens if someone wrote the pain down last time. Skip the retro and next loop starts from zero.

> **Field note — from a real project:** *CI/CD Pipeline Excellence, cashed in from three retros of notes.* Eighteen issues across six layers — none of them invented fresh, all of them traced back to retro lines from earlier releases that said "we should fix this next time." The retros were the ledger. The release was the settlement.

Rule: Cycles nest. Each level needs a theme, a boundary, and all three phases.

Why: A loop at the wrong scale is either too small to ship anything meaningful or too big to retro honestly. And at every scale, an upfront plan, an execution phase, and a validation phase all have to be present — compress any one of them and the cycle stops being legible. A cycle without a theme gets named by its date and forgotten. A cycle without a shippable boundary can't have a retro because you can't tell what "done" looked like. A cycle without explicit validation ships vibes. **How to apply:** The task-level loop is §4 as written — brainstorm, plan, TDD, verify, commit, retro. The release-level loop is the same shape on a larger canvas: a themed slice of the roadmap, an upfront plan (often a file map for parallel work), execution (often in waves), validation in the environment that matters, and a retro written in voice with a nickname. The project level is the same again: a PRD that sets the arc, a roadmap, a final retro. Each level needs a name you could say in one sentence and a boundary someone outside the work would recognize as shippable.

The second case study is the exemplar. Nine releases, each a themed slice of the same PRD, each with a clean boundary, each with a retro that reads like a story — every release title a thesis, every thesis a cycle you could retro honestly. The themes *are* what made the retros writable — a named theme wants to be a story; "Sprint 23" doesn't — and the retros are what made each next release plannable, because the lesson from one cycle became the opening move of the next. The wave pattern from an early release ("trivial work first, to shake out the build/config/test harness before committing agents to heavyweight implementations") became standard practice for every release after it. That transfer only happens if the cycle had a name worth remembering.

The negative example is the Onion release from the first case study — six layers of CI failure peeled one at a time over a week, with no theme, no single thing you could name the cycle for. *That* is why the layers were able to hide inside one release: the cycle had no shape, so nothing about the retro-able boundary forced the hidden work out into the open. Themed cycles aren't a ceremony; they're how the work becomes legible enough to improve.

> **Field note — from a real project:** *Nine retros that taught the tenth.* The second case study's retro practice is the proof of concept for this rule. Each release was themed; each theme was one sentence; each retro had a "what we learned" section that became the next release's plan. By release nine, the wave pattern, the zero-dependency posture, the merge-point discipline, and the descoping rules were all things the team had written down *to themselves* in earlier retros and then honored in the next cycle. The cycles nested because each level had a theme, a boundary, and a lesson worth carrying forward. *Pin: name the cycle before you start it; retro it the moment it closes.*

Part II — Working Together

§5 Memory hygiene

Part I was how to start the first conversation. Part II is how the partnership survives more than one. Memory is the first chapter because it's the persistence layer — what makes §4's loop stick from Tuesday to Thursday. Get it right and next week's agent shows up already knowing the scars. Get it wrong and every Monday is a cold start with confident wrong answers.

Rule: Memory has four types. Use the right one or it rots.

Why: User facts, feedback, project state, and reference pointers have different staleness profiles. Mixing them produces a junk drawer where nothing is trusted because some of it is always wrong. **How to apply:** User preferences → user memory. Corrections from past conversations → feedback memory. Sprint cycle, current version, live project state → project memory. Pointers to Linear, a retro, an external doc → reference memory. Write the type at the top of every memory file so the next reader knows what they're looking at.

The failure mode is one big memory file holding "the user prefers parallel agents" two lines above "current sprint cycle ID is 3827cdb9" two lines above "always run the notification sound after tasks." Three half-lives, one file, no cleanup schedule. The preference is good for years. The cycle ID is stale in six weeks. The sound-notification rule belongs in the harness config, not memory at all. When a file mixes all three, nobody prunes it — pruning means re-reading everything to decide what's still true, and nobody has the afternoon. Separate the types and each gets its own small, obvious pass.

> **Field note — from a real project:** *CLAUDE.md Gets a Haircut, the four-way sort.* The 60% pruning pass turned out to be a sort operation in disguise: stable facts to topic files, live state to memory, procedures to skills, permissions to harness config. Every line in the old file belonged in exactly one of those four places. The reason the file had grown so large was that nobody had ever been forced to decide which one.

Rule: Stale memory is worse than no memory.

Why: An agent with no memory asks. An agent with stale memory acts — confidently, on last quarter's facts, with no tell that anything's wrong. The lookup the first agent would do is the lookup the second one skips. **How to apply:** When you read a memory line and it's wrong, fix or delete it in the same turn. Not "I'll clean that up later." Later is the bug. Budget a memory pass at the end of every release, with a bias toward deletion.

The Four Scoping Gaps story is the canonical version and it cost us rounds. A memory line from the previous release said the release-automation tool handled downstream workflow triggering. By the next release we

knew better — that tool plus the default workflow token doesn't trigger downstream workflows at all — but nobody had gone back to fix the note. A subagent picked up a release task, read the memory, acted on it, and we burned a round on "why didn't deploy fire?" before someone remembered the note was already wrong. No memory would have forced a fresh doc check. Stale memory hid the check behind a sentence that looked authoritative. Cheap to fix when you notice; expensive every round after.

> **Field note — from a real project:** *Four Scoping Gaps, half of them stale memory.* Two of the four gaps that tanked the release were old mental models we'd never written down as corrections. A subagent with a memory line that still said "the tool handles downstream triggering" cannot rediscover that it doesn't, because the line is already answering the question the doc check would have raised. Stale memory is the lookup the fresh agent would have done, deleted.

Rule: Save the why, not just the rule.

Why: A year from now, "always use X" is unfollowable when X conflicts with the new architecture. "Always use X because Y bit us in v4.0" lets future-you decide whether the rule still applies or whether the world moved on. **How to apply:** Every feedback memory needs a **Why** line and a **How to apply** line. Always. If you can't write the why, you don't understand the rule well enough to save it — find the scar before you file the note.

Context-free rules become dogma. The Cache That Lied In CI is the example we still cite (full story in Part V). The rule "use `.model_dump(mode='json')` before caching Pydantic models" was written down. The *why* was not. Months later a new agent reached for `json.dumps(model, default=str)` because it looked equivalent, and the image-generation pipeline broke in CI while passing locally. With the why attached — "because `default=str` calls `str()` and produces a repr, not a dict" — the next person knows which footgun the rule is aimed at. Without it, you're carrying a superstition with no target.

> **Field note — from a real project:** *The cache that worked locally and lied in CI.* A refactor ran Pydantic models through Redis using `json.dumps(model, default=str)`. Local was green because local used an in-memory cache with no serialization path at all — the model went in as an object and came out as an object. CI had actual Redis, which needed JSON, which exposed that `default=str` calls `str()` and produces a Python repr, not a dict. The rule that went into the codebase was "use `model_dump(mode='json')`." The rule that should have gone into memory alongside it — with a *why* attached — was the second half of the story.

Rule: Don't memorize what the code already says.

Why: File paths, function names, module boundaries, route registrations — all one `grep` away. Memory is for what's *outside* the repo: preferences, corrections, live state, cross-system gotchas. Duplicating what the code says is how memory silently disagrees with reality the moment someone renames a file. **How to apply:** If you can find it with one grep, don't put it in memory. If it lives in `git log`, don't put it in memory. If a test asserts it, double-don't — the test is already the source of truth, and the memory line will drift out from under it.

The CLAUDE.md haircut is this rule enforced at scale. The file had grown to hundreds of lines restating what the code already made obvious: the directory tree, every endpoint path, which file held the admin router, how the audit middleware was wired. All of it was `ls` and `grep` away. We deleted it in one pass and the agent got *better* at finding things — because now when it needed a fact it read the code instead of trusting a note that had been quietly wrong for a month. Memory carries what the code can't tell you. Everything else is load the code can carry itself.

> **Field note — from a real project:** *CLAUDE.md Gets a Haircut, the speed paradox.* After a 60% cut, work got *faster*, not slower. Every line that came out was either duplicating what a `grep` would have found or asserting something the code had quietly diverged from. Removing them meant the agent started reading the code when it needed a fact, which was more reliable than reading a stale note and *also* faster than reading an 900-line preamble on every cold start.

§6 Skills as institutional knowledge

§5 was about pruning what you remember. This chapter is about what you shouldn't trust yourself to remember at all. A **skill** is a procedure with discipline: invoked on purpose, executed step by step. A **CLAUDE.md note** is a fact you hope gets followed. Only one of those is load-bearing when you're tired. *Skills are followed; notes are remembered — and the second one is a lie.*

Rule: Skills are procedures with discipline. CLAUDE.md notes are facts you hope get followed.

Why: Invoking a skill is a deliberate act — the agent opens it, reads the steps, runs them. Reading CLAUDE.md is passive ambient context, absorbed on cold start and then competing with everything else in the window. The difference is whether a step happens because the harness made it happen or because the agent felt like it this turn. **How to apply:** If a procedure has more than three steps and matters, write it as a skill. If it has three or fewer and matters, write it as a skill anyway. The threshold isn't length — it's whether you want it *executed* or merely *recalled*.

The tell is the CLAUDE.md paragraph that starts "remember to also…" The agent doesn't *remember*; it re-reads on cold start and decides, turn by turn, which lines are relevant. Anything that must happen reliably needs a host other than the agent's attention. Release flow is the canonical case: a CLAUDE.md line notes where version lives, and a release skill runs the sync script, drafts the retro, walks the deploy. Note is atmosphere; skill is action. When we confused the two in the Four Scoping Gaps release — treating "the release-automation tool handles downstream triggering" as a remembered fact rather than a procedure step — we burned a round on a Release PR that merged cleanly and triggered nothing. *Pin: if you want it done, skill it. A note is optional.*

> **Field note — from a real project:** *Four Scoping Gaps, half of them procedures living as notes.* Two of the four gaps that tanked the release-automation change were procedure steps that had been filed as CLAUDE.md notes — "the tool handles the downstream trigger" and "target-branch goes in the config." Both were half-true and the untrue halves were exactly what a skill checklist would have forced the agent to verify before moving on.

Rule: Rigid skills exist for a reason. Don't adapt the discipline away.

Why: TDD, systematic debugging, brainstorming-before-planning — these skills are rigid because the failure mode they prevent *is the failure to follow them.* They feel like overkill in exactly the moments they're needed most, because the same thing that makes them feel unnecessary — "the problem looks simple" — is why you're about to skip them. Simple problems are where rigid skills earn their keep. **How to apply:** Follow rigid skills exactly, especially when you're sure you don't need to. Six steps means run six. If you genuinely skip one, skip it *out loud* — same rule as §4's loop.

The Four Scoping Gaps release is the version of this lesson I'd like to forget. The brainstorm skill would have forced us to ask *what exactly does this tool read, and from where?* before we wrote a line of config. We skipped it because the fix was "obvious": add one config line, flip a checkbox, done. It turned out to be four fixes in a trench coat (full catalog in Part V). A fifteen-minute brainstorm catches all four. We skipped it because we were sure we didn't need to. *Pin: when the rigid skill feels like overkill, run it twice.*

> **Field note — from a real project:** *Four Scoping Gaps, skipped brainstorm edition.* The "obvious fix" was one config line. The real fix was four separate changes in three systems. Fifteen minutes of brainstorming — the exact ritual that felt like overkill — would have surfaced every one by asking "what does this tool actually read?" We skipped it because we were sure we didn't need to.

Rule: Write a skill after the third correction.

Why: The first correction is a one-off. The second is a pattern. The third is the moment "I keep saying this" becomes "the system should enforce this." Promoting earlier wastes a skill slot on a fluke; promoting later means you've spent a week re-typing the same sentence. Three is the elbow of the curve. **How to**

apply: Track your corrections. When you catch yourself writing the same nudge a third time, promote it — not to a CLAUDE.md paragraph, which is back to passive notes, but to an actual skill or a harness hook. The goal is to stop needing the correction at all.

The Trust Your Local Tests turnaround is this rule made visible. For three releases running we'd been typing the same correction into fresh subagent conversations: "Postgres only, no SQLite branches, trust your local tests." By the third correction the cost wasn't the typing — it was the fear crystallized around "local tests might be lying." The promotion move was to rip the SQLite branch out of the test bootstrap, wire the fixtures to a shared Postgres engine with per-test truncation, and bake the agreement into the infrastructure. "Trust your local tests" is no longer a note the next agent has to remember; it's the only thing the code lets them do. The correction became structurally unnecessary. Third time you correct the agent the same way, the system should change, not the prompt. *Pin: count your corrections. Three is the promotion line.*

> **Field note — from a real project:** *Trust Your Local Tests, three corrections in.* The phrase had been typed into fresh subagent conversations for three releases running before someone admitted that another prompt correction wasn't going to fix it. The promotion wasn't another memory line; it was infrastructure that made the old mistake impossible. When the third correction hits, change the system.

Counter-case — when not to promote. Promotion has a cost: a shared abstraction is code that every consumer becomes coupled to, and abstractions over code that isn't *actually* duplicated yet are worse than the duplication they claim to fix. The second case study hit this explicitly. When a later release added a second consumer of a binary-encoding helper, the plan called for ~200 lines of helpers copy-pasted from the existing caller — because the only other caller was one directory over, the helpers were small and stable, and a shared package would have forced both consumers into the same abstraction at exactly the moment the second one was still finding its shape. The retro named it directly: *"decided the duplication was less disruptive than the abstraction. Ask us again in a year."*

The fourth case study has both halves of this story, paired across two releases. **Extract worked when the pattern was stable:** one release had 825 lines of duplicated edge function boilerplate across 15 functions — CORS handling, auth checks, rate limiting, error shaping, response formatting, all repeated 15 times with subtle differences. The pattern was *stable* across all 15 callers; nobody was inventing new variants. Extracting it into a 139-line shared handler pipeline was a clean win and immediately caught two functions that had silently *omitted* a step the pattern guaranteed. **Failure-to-extract bit them when the duplicates drifted:** the same project let a small `naivePlural()` helper get duplicated across three different edge functions instead of extracting it. By the next release, one of the three copies had incomplete irregular-noun handling — missing `tooth`, `goose`, `child`, `person` — so the same input produced *different* outputs depending on which function rendered it. The user-visible bug was "1 mouse" rendering as "1 mice" on one path and "1 mouses" on another; the root cause was three copies that had drifted apart while nobody was looking.

The rule for promotion isn't "count to three and extract" — it's *"count to three and then check whether the abstraction is cheaper than the duplication for the shape the code is in right now, and whether the duplicates will drift if you don't."* Sometimes the third correction is a skill. Sometimes the third copy is still cheaper than the shared thing. Sometimes the third copy is *already drifting* and you should have extracted at the second. The test has two parts: (1) is the duplicated logic still *finding its shape* (duplicate is fine) or has it *stabilized across callers* (extract); and (2) are the copies drifting silently from each other (extract regardless of stability — drift is the more dangerous failure). *Pin: promotion is a bet that the pattern has stabilized. Don't make the bet until it has — but don't refuse the bet so long that the duplicates start lying to each other.*

§7 Verification before completion

If you take one chapter of this playbook seriously, take this one. Verification is the load-bearing chapter — every other discipline in Part II points back here, because every other discipline fails the same way when verification is sloppy: confidently, in production. The rules are dry; the consequences are not.

Rule: "Tests pass" is not "feature works." Verify the feature.

Why: Tests verify the code under test, not the user-visible behavior of the system. A suite can be 100% green while the feature is broken because no test exercised the wire between the parts. **How to apply:** For UI work, open a browser. For deploy work, hit the live endpoint. For data work, query the data. For API work, read the response *body*, not just the status code. If "done" lives behind a screen, look at the screen.

The Doorbell That Never Rang is the cleanest version of this we have (full story in Part V). We shipped audit logging with backend tests green, frontend tests green, and a beautiful admin page that displayed audit events. There were zero events in it. Backend exposed the read endpoint. Frontend rendered the list. Nothing in between actually emitted an audit event when an admin did anything. Both halves were tested in isolation; nobody wired the doorbell to the button. No test exercised the path from "admin clicks" to "row appears in the audit table." A thirty-second click in a real browser would have. *Pin: if you didn't open the thing and use it, you didn't verify it.*

> **Field note — from a real project:** *The Doorbell That Never Rang.* Audit logging shipped with every test green and zero rows in the live audit table. Backend exposed the read endpoint. Frontend rendered the empty list beautifully. No service call anywhere in the admin routes actually emitted an audit event. Both halves were tested in isolation; the wire between them was never built. A thirty-second click in a real browser would have caught it on day one.
>
> **Field note — from the same project, the inverse failure:** *The Comparison That Quantum-Superposed Into Nonexistence.* A "Comparison of the Day" feature shipped to the homepage. The component was implemented correctly, the data fetched correctly, the unit tests passed, the integration tests passed, the deploy was clean. And it never appeared on the page. Root cause: the component was wrapped in `{!isLoading && <Feature />}`, and on the homepage's particular data path `isLoading` never resolved to false because of a stale-cache race condition in a sibling component. The feature rendered into a virtual DOM nobody ever painted to the screen. The Doorbell story is about *missing wiring* — the audit event never fired. This is the inverse: *the bell rang, perfectly, into a room nobody could see into.* Same root class — "tests pass, feature broken" — completely different mechanism. The Doorbell would have been caught by a real-browser click. So would this one — *if* the click happened on the page where the bug lived. The deeper lesson stacking across both stories: **a green test suite tells you the parts work; only a real session in the real browser, on the real page, with the real data, tells you the user can use the thing.** Two failure modes, one rule, two reasons to verify in the environment that matters.
>
> **Field note — from the second case study:** *The bufconn Gap.* One adapter had full unit test coverage using an in-memory connection that bypassed network transport — and every test passed beautifully. The first smoke test from a real client against the running server discovered that a required discovery feature had never been enabled, and the client had no way to find the service. The fix was two lines. The lesson was older than the protocol itself: **your tests can only catch the things they actually test.** An in-memory loopback tests *logic*. It does not test *deployment*. A complete test suite against the in-memory path is not a complete test suite against the wire. The retro's one-line extract: *"this is not an argument against unit tests; it is an argument for also just… running the thing."*

Rule: Evidence before assertions. Always.

Why: An agent that confidently says "the build passes" without showing the output is the agent you cannot trust. Verification is a habit, not a claim. Accept "I ran the tests and they passed" as evidence and you've trained the agent that prose is sufficient — and prose is what hallucinations look like. **How to apply:** Never claim success without producing the verification command *and* its actual output, pasted in. `pytest` plus the green dots is evidence. "Tests pass" is not. If the agent says "deploy succeeded," the next sentence had better be a `curl` against the live URL with the response body attached. *Sharpening:* evidence of the *wrong claim* is still theater. A green check is only evidence for the specific claim it tests. Before accepting any successful output, name the claim the evidence actually supports. "180 unit tests passed" is evidence

that 180 unit tests passed — it is not evidence that the feature works, the component renders, the cache-hit path fires, the touch event reaches the handler, or any of the other claims the green check might be mistaken for. The third case study's v7.0.1 hotfix is the canonical example: a data-layer migration shipped with a full green suite, and the failure was in the cache-hit path no test exercised. The tests weren't lying; the humans were reading them as answering a question they had never been asked. Evidence that answers the wrong question is not weaker evidence — it's *theater*, and it's dangerous precisely because it *looks* authoritative.

The Verify-What-You-Shipped scar is canonical (full story in Part V). The deploy script reported success, the new container had started, every status check was green — and the live site was still serving the old version, because nginx had never flipped to the new container. "I started the container" had become a stand-in for "the container is reachable from the internet." The fix was one line: after the flip, `curl` the real public health URL and check the response body for the new git SHA. Not SSH exit codes, not unit status — the user-facing URL, the actual body, compared against the SHA we just built. *Pin: the verification command and its output, pasted, or it didn't happen.*

> **Field note — from a real project:** *Verify what you shipped, not what you built.* The deploy script reported success. The new container was healthy. Every status check was green. The live site was still serving the old version because an orphan container from the previous compose definition was holding the port and nginx had never flipped. "I started the container" had become a stand-in for "reachable from the internet." The fix was one curl against the real public URL, comparing the response body to the SHA we'd just built.

Rule: Verify in the environment that matters.

Why: Local-passing + CI-failing is the single most expensive failure mode in this project's history. Cache backends differ. Databases differ. Node versions differ. Serialization paths differ. "Local green" only proves the code works against the specific lies your laptop tells. **How to apply:** If the change touches anything that runs in CI or production, run it there before claiming done. If you can't reproduce CI's environment locally, that's the bug — fix the parity gap, then verify. *Cross-reference:* this is the same lesson the DevOps playbook pins as "dev == prod." Two playbooks, one rule, and the overlap isn't an accident — parity between environments is *the* substrate that makes verification mean anything. If you have the DevOps discipline, this rule is cheap. If you don't, this rule is where you find out.

The Cache That Lied In CI is the worst version we shipped (full story in Part V). The cache layer got refactored, Pydantic models started flowing through Redis, someone wrote `json.dumps(model, default=str)`, the suite was green locally, and the image-generation pipeline exploded in CI. Local was green because local used an in-memory cache with no serialization path at all — the model went in and came out as an object, nothing ever called `json.dumps`. CI used Redis. The serialization step that didn't exist locally was the entire bug. A later release generalized it into a standing rule: any cache-related change passes locally and fails in CI until that parity gap closes. The fix is `model_dump(mode="json")`; the rule is bigger. *Pin: if your local doesn't run what CI runs, your local green is a lie.*

> **Field note — from a real project:** *The Cache That Lied In CI, the parity gap that followed.* The fix for this specific bug was one serializer call. The *rule* that came out of it was bigger: any cache-related change passes locally and fails in CI until local and CI share a serialization path. That rule was carried open as a tracked parity gap through three releases, and finally closed when a separate cleanup ripped SQLite out of the test suite in the same move — closing two parity gaps with one piece of infrastructure.

> **Field note — from the second case study:** *The Docker Port Mappings That Weren't.* One hundred and eighty passing tests. Zero data races. Four clean releases. And a Docker container that had been completely unreachable from outside for every single one of those releases, because the Makefile's port mapping routed a host port to a privileged container port — and the binary inside the container ran as a non-root user that could not bind privileged ports and was listening on a different port entirely. Every port mapping was a Potemkin village pointing at an empty socket. The first manual smoke test against the container returned an immediate TCP RST. CI had validated that the image *built*. CI had never validated that the image *answered*. The fix was

fifteen lines across three files. The retro's one-line extract: *"a test suite with 180+ passing tests and zero race conditions can still ship a product that doesn't respond to a raw connection. Unit tests prove your code works. Integration tests prove your components work together. But nothing proves your deployment works except deploying it and poking it with a stick."* The deeper lesson is the same as The Health Check That Wasn't, from a completely different project: **test what you ship, not what you build.** A successful `docker build` tells you your binary compiles. It tells you nothing about whether the mapped port reaches the process. Two projects, two languages, two different infrastructure stacks, one scar.

Field note — from the third case study: *The TanStack Query Migration That Broke Cache-Hit Paint.* A time-tracking web app migrated its entire data layer from manual `useEffect/useState` fetching to TanStack Query — every hook, every cache path, every mutation. 161 unit tests passed. The E2E suite passed. The PR merged and shipped to production. Within 24 hours a user reported that painting didn't work. Root cause: the migration had moved `setActiveCategory(rows[0])` inside the TanStack Query `queryFn`. With `staleTime: 5min` and `refetchOnWindowFocus: false`, TQ serves cached data *without running the queryFn* on most loads — so `activeCategory` stayed `null` and every paint interaction silently no-op'd. The fix was five lines: replace the side effect with derived state via `useMemo`. The lesson the retro named: *"never put React state mutations inside a queryFn. TQ runs on TQ's schedule, not React's."* But the deeper lesson is the one this rule is about: **infrastructure migrations need tests that exercise the interaction pipeline, not just the data pipeline.** Unit tests validated the queryFn ran correctly on cache miss. Nothing in the suite exercised "first page load → cached categories → user clicks cell → paint fires." The tests were green *for the claim they tested*, and the claim they tested was not "the feature works." A single Playwright test with `page.click()` against a pre-warmed cache would have caught it in five seconds. When you're swapping a foundation layer — data layer, auth layer, router, state management — the test gap is always on the interaction side, because the unit tests were written for the *old* substrate's assumptions.

Field note — from the third case study: *The iPad Tap That Wasn't.* Shipped in the same release as the TanStack Query migration, for the same reason: a touch-vs-mouse abstraction that had been written and tested on desktop only. The grid's `handlePointerDown` had a guard — `if (isTouch && !paintModeEnabled) return` — designed to prevent drag-to-paint from hijacking scroll. On iPad Safari this guard also blocked *single taps*, which don't conflict with scroll at all. Users tapped cells and nothing happened. The fix was to allow touch `pointerdown` to fire the handler unconditionally, block only `pointerEnter` during touch scroll, and add an `onPointerCancel` handler to reset paint state when iOS took over the gesture. The explicit retro lesson: *"input method abstractions (touch vs mouse) should be as narrow as possible — guard the specific conflict (drag-to-scroll), not the entire input channel."* The implicit lesson, stacked on top of the TQ bug: two hotfixes in one release, both from the same gap — the test suite never exercised the interaction pipeline on the actual target device. The two hotfixes together are the meta-lesson the retro names: *"migration testing should cover interaction paths, not just data paths."* Run the actual gesture on the actual device through the actual new substrate, or you're verifying a claim you weren't asked about.

Rule: Health checks must check health, not "is the process running."

Why: A 200 from `/health` doesn't mean the database is reachable — it means nginx is awake. A health check that can't detect a downed dependency is a status check cosplaying as a health check, and the worst thing it can do is succeed during an outage. **How to apply:** Health checks must verify the actual dependencies — DB ping, Redis ping, downstream reachability — and return them in the response body, with the build SHA, so verifiers can confirm they hit the right endpoint and the right version. If it can't tell "healthy" from "half the stack is on fire but uvicorn is still up," it isn't one. *(See DevOps Playbook Phase 7.1 and Gotcha #8 for the `/health` vs `/api/v1/health` split and the verification pattern that catches this.)*

The Health Check That Wasn't is the canonical field note for this rule and probably for the whole chapter.

We believed for three deploys that `git_sha` kept coming back `unknown` after release because the build-arg plumbing was dropping it somewhere in the YAML. We chased the producer through every Docker mechanism we had. Three rounds in, somebody dumped the raw response from the verification curl. The endpoint was `/health` — which returns `{"status":"ok"}` and has never had a `git_sha` field. The endpoint with the full health JSON and dependency checks was `/api/v1/health`. We spent three deploys fixing a producer that was never broken because the verifier was hitting a status check pretending to be a health check. The lesson is twofold: health checks must actually report health, and verification curls must hit the endpoint that reports it. *Pin: if your health check can't say what's broken, it can't say anything's healthy.*

> **Field note — from a real project:** *The Health Check That Wasn't.* Three deploys spent fixing a producer that was never broken, because the verification curl was hitting a two-line status endpoint instead of the full health endpoint — the status endpoint has no `git_sha` field and never did. The lesson lives in two rules at once: health checks must actually report dependencies, and verification must hit the endpoint that reports them.

§8 Trust boundaries

Verification (§7) tells you the work is correct. Trust boundaries tell you whether the agent had the authority to do the work at all. This chapter is the political layer on top of the technical one. Get the technical layer right and the political one wrong and you ship a verified force-push to main.

Rule: Match the action to its blast radius. Confirm before crossing the line.

Why: A local file edit is reversible. A force-push is not. Treat both the same and you'll eventually force-push something that should have been a file edit. The agent doesn't intuit blast radius — it sees both as "a tool call I'm allowed to make." That intuition is the engineer's job. **How to apply:** Local and reversible — let the agent run. Shared state, hard to reverse, visible to other people — confirm first. The test is recovery time: if this is wrong, how long to undo it? Seconds, fine. Hours of someone else's work, ask first.

The cost of an action is not visible in its syntax. `git commit` and `git push --force-with-lease origin main` are two characters apart and four orders of magnitude apart. The Four-Fix WCAG Contrast Cycle is the calibrated version of this (full story in Part V) — four consecutive contrast violations across two releases, each fix a Tailwind class swap of one shade. Reversible, local, tiny blast radius. The right move was to let the agent change the colors, run the axe-core suite, and confirm by *evidence* (§7) rather than by approval — the verification step did the blessing. A deploy in the same session gets a manual confirmation, because the recovery time is "how long until rollback finishes," not "Cmd-Z." Different blast radius, different boundary. *Pin: blast radius is set by recovery time, not by command length.*

> **Field note — from a real project:** *The fourth contrast fix in two releases.* Four WCAG contrast violations across two releases, each a one-class utility color swap caught by axe-core on the next run. Reversible, local, tiny blast radius. The right move was to stop adding approval ceremony to the fixes and let the automated rail do the blessing — the rail was doing its job; humans were the slow learners.

Rule: Authorization is scoped, not blanket.

Why: "Yes, push" once does not mean "yes, push" forever. When a user approves a destructive action, they are approving *that specific action*, not handing out a permission slip for the next one. An agent that generalizes a single yes into a standing yes will eventually cross a line nobody told it not to cross. **How to apply:** Every risky action is its own decision unless durable instructions — CLAUDE.md, `settings.json` permissions, an explicit skill — say otherwise. If in doubt, ask. The cost of asking is one round-trip. The cost of not asking is a retro chapter.

The interesting code, as the Admin Who Couldn't Fire Herself release put it (full story in Part V), is in the *denial* path. RBAC went in for admins, and the rules that needed care weren't "an admin can edit a record" — those were one-liners. The careful rules were "no self-role-change, no self-deactivation, no self-deletion, no escalation via update." Half a dozen explicit refusals, because trust granted in one direction (you are

an admin) does not generalize in every direction (therefore you can fire yourself). Scoped authorization for agents works the same way. You approved a `git commit`; that does not mean the next `git push --force` is pre-approved. Where the harness can express "this tool, this scope," use it — harness permissions are the durable form of "yes, but only this." Everything else is a per-action decision. *Pin: every yes is for one action. The next risky action is its own conversation.*

> **Field note — from a real project:** *The Admin Who Couldn't Fire Herself.* RBAC landed for the admin panel and the one-liners were all in the "yes" paths. The careful rules were in the refusals — no self-role-change, no self-deactivation, no self-deletion, no escalation via update. Half a dozen explicit denials, each written on purpose, because trust in one direction doesn't generalize in every direction. A later release added one more the first pass missed: the last superadmin can't fire themselves either.

> **Field note — from the same project, the corollary:** *The Default That Was Admin.* A later release shipped a refactor of the edge function pipeline that introduced a shared handler signature. The signature accepted an optional `client:` parameter — `'admin'` or `'anon'` — and **defaulted to `admin` if you didn't specify**. Existing functions worked fine because they had been written to pass the right client explicitly. The bug was in *new* functions: every new endpoint a developer wrote had admin privileges by default, because *not thinking about the client at all* meant getting the admin client. A security review in the next release caught two endpoints that should have been anon (a public stats endpoint, a public leaderboard) running with admin keys for a full release cycle. No data leaked because the underlying RLS happened to cover it, but the privilege gradient was inverted. The fix was one line: change the default to `anon`. The lesson is the corollary to "authorization is scoped, not blanket": **the default direction also matters. The thing you don't think about is the thing that needs to be the safe choice.** A default is the part of the design that catches every developer who didn't read the docs — including agents, whose default behavior is "use the API the way the type signature suggests." Defaults must be the least-privilege option, every time. *Pin: the unsafe default is the one that gets used by everyone who didn't think about it. Make the safe one the default; require explicit opt-in for elevation.*

Rule: When the agent hits an obstacle, it must investigate, not delete.

Why: Unexpected files, branches, lock files, and orphaned containers usually represent in-progress work — from a teammate, from an earlier you, from a process that hasn't finished. Deleting them to make the error go away is how you lose a day of uncommitted work nobody can reconstruct. **How to apply:** Investigate root causes. `git reset --hard`, `rm -rf`, `--no-verify`, `git clean -fd`, "let me just drop the database" — last resorts, not first resorts. If the agent cannot tell you *why* deleting the thing is safe, it isn't.

The Deleting the Ghost near-miss was a deletion that almost shipped (full story implied across Part V). The release was cleaning up a legacy deploy script — replaced by the new blue-green version, no callers in the codebase, no references in any workflow, every grep clean. By every test the agent could run, the file was dead code. The code-quality reviewer caught it: the operations runbook still pointed at the old script as the manual recovery path during a rollback. The reference wasn't in any source file or workflow — it was in a Markdown runbook nobody had thought to grep. One careless deletion would have left the on-call playbook pointing at a ghost. The rule generalized: never delete a file without grepping for its name across docs and runbooks first, and when the agent says "this is safe to delete," the right response is "show me where you looked." *Pin: if the agent can't show you where it looked, it didn't look.*

> **Field note — from a real project:** *Deleting the Ghost.* A legacy deploy script was scheduled for deletion. Every grep of the source tree and workflows came back clean. A code-quality reviewer noticed the operations runbook still pointed at the old script as the manual recovery path during a rollback — a reference that lived in a Markdown file nobody had thought to search. Deleting it would have left the on-call playbook pointing at a file that no longer existed. Grep your docs before you delete anything.

Part III — When It Goes Wrong

§9 Failure modes & recovery

If you're reading this first, something is on fire. That's fine — this is the door we left unlocked for that. Five failure modes follow, easiest to recognize first, most dangerous last. Find the one that matches, read what's happening underneath, do the fix. We've been in all of these. None is terminal.

Failure mode: The agent went off the rails

Symptoms: Code that has nothing to do with what you asked for. Files invented out of thin air. The agent solving a problem you don't recognize. You read the diff and think "what conversation were we even having?" **What's actually happening:** The brief was vague or missing, and the agent filled the context gaps with guesses. Every prompt is a cold start (§1); when the prompt doesn't say what "done" looks like, the agent invents a "done" that sounds reasonable and ships toward it. **The fix:** Stop. Don't steer mid-flight. Open a fresh conversation and write a self-contained briefing — goal, constraints, what you've already ruled out, success criteria. See §3 The first conversation.

The instinct is to send another message correcting the drift. Don't. A one-line correction to a context-starved conversation gives the agent a new way to be wrong. Kill it, write the briefing you should have written the first time, start over. You're throwing away ten minutes of nonsense to save an afternoon of it.

Failure mode: The agent produced slop

Symptoms: Code that compiles but feels wrong. Generic helpers nobody asked for. Defensive try/except around errors that can't happen. Three layers of abstraction over a function called once. The PR is twice the size it should be and reads like a coding-interview answer. **What's actually happening:** No constraints in the prompt. The agent's default mode is to add — a new helper, a new layer, an extra branch — because adding feels like work and looks like care. Without a "don't add features beyond the task" rail, every prompt drifts toward gold-plating. **The fix:** Add explicit YAGNI constraints to the prompt. Better, make them durable: promote "no abstractions for a single caller; no handling for impossible errors; no refactoring adjacent code unless asked" into memory or a skill. See §5 Memory hygiene and §6 Skills as institutional knowledge.

The morning-after tax is the tell. If you've spent a morning removing what the agent added yesterday — the unused helper, the wrapper, an early-return for a case that doesn't exist — that's slop. The God Module Problem hits the same pattern at the file level: god modules grow because nothing justifies the interruption to split them. Slop grows the same way. Constrain at the prompt; promote to memory the second time you re-type it.

> **Field note — from a real project:** *The God Module Problem.* One admin module grew to 1,052 lines because every new feature "just added one more route to the existing file." Nothing ever justified the interruption to split it. The cost wasn't visible until someone tried to brief a subagent on "admin behavior" and had to load the whole thing. Slop grows for exactly the same reason — each addition is too small to argue with.

Failure mode: The agent can't be trusted with anything important

Symptoms: Every output needs heavy review. You're editing the agent's work more than you're working alongside it. You stopped delegating things that matter because checking costs more than doing it yourself. Velocity collapses; morale follows. **What's actually happening:** No verification discipline. The agent has been rewarded for sounding done rather than being done — every "looks good, ship it" without an evidence check trains the next "tests pass" to be a vibe instead of a fact. Trust didn't erode; it was never built on anything. **The fix:** Install §7 Verification before completion, hard. Evidence before assertions, every time. Make the verification command and its output part of the deliverable, not a courtesy.

This feels like a people problem and isn't — the loop lacks a forcing function. The Trust Your Local Tests pivot is the cleanest version: three releases of typing "trust your local tests" into fresh conversations, each correction evaporating by the next session. The fix wasn't another correction; it was ripping SQLite out of the suite so local and CI ran the same paths. You can't out-discipline a missing rail. Build the rail.

> **Field note — from a real project:** *Trust Your Local Tests, the discipline problem that wasn't.* Three releases of the same correction taught us that "try harder" was never going to work. The rail was missing. Building the rail was an afternoon; living without it had already cost us a quarter.

Failure mode: The agent loops on the same wrong fix

Symptoms: The agent fixes something, the test fails, the agent fixes it differently, the test fails the same way. Each fix is plausible. None work. You're three rounds in and the error message hasn't moved. **What's actually happening:** Misdiagnosed root cause. The agent is iterating on a problem that isn't the problem. The fixes look reasonable in isolation; none touch the actual broken thing because nobody asked "wait, what is actually broken?" **The fix:** Stop touching code. Run systematic debugging — dump raw state, log the actual response, walk the data path from producer to consumer. Find the real failure before the next edit. See §7.

The Health Check That Wasn't is the canonical example — three deploys spent fixing a producer that was never broken, because the verification curl was hitting the wrong endpoint. Full story in Part V; what matters here is the move: when the second fix fails the same way as the first, stop fixing and start diagnosing. *Pin: when the second fix fails the same way as the first, stop fixing and start diagnosing.*

> **Field note — from a real project:** *The Health Check That Wasn't, the debugging lesson.* Three deploys, three plausible fixes, zero movement in the error. The break came when someone stopped editing and dumped the raw response — the question "what's actually in this payload?" had never been asked out loud. When the second fix fails the same way as the first, the root cause is misdiagnosed. Stop touching code and start looking at data.

Failure mode: The agent confidently lies about state

Symptoms: "Tests pass." (They don't.) "Deploy succeeded." (It rolled back.) "I verified the endpoint." (It returns 404.) The summary is fluent, optimistic, and wrong. You only catch it because something downstream breaks an hour later. **What's actually happening:** No evidence-before-assertions discipline. The agent learned that a confident summary closes the conversation, and "I ran X and it passed" reads identically to "I ran X and pasted the output." Every other failure in this catalog announces itself. This one looks like progress and rots underneath. **The fix:** Install §7 *very* hard. Reject any success claim not accompanied by the verification command and its actual output. Where the harness can enforce it — hooks, settings.json, skill scaffolding — make evidence mandatory, not optional.

Failure mode: The partnership architecture is wrong

Symptoms: The agents are doing competent individual work. Each one's output looks clean. Tests pass on each piece. Reviews are clean. And yet the integration is a mess — files end up on the wrong branches, commits cross-contaminate, state leaks between tasks that were supposed to be isolated, or work that was supposed to compose silently conflicts with itself. When you go looking for "the bug," there isn't one — every agent did exactly what it was told, and the failure emerged from the way the agents were *arranged*, not from anything any single one of them did. You find yourself debugging topology instead of code. **What's actually happening:** This is the failure mode every other entry in this catalog misses. The first five are all discipline failures — no brief, no YAGNI, no verification, no root-cause, no evidence. Those are real, but they're not the only way the partnership breaks. Sometimes the agents have *all* the discipline and the *architecture* of how they're collaborating has a hidden incompatibility with the underlying tools. The third case study hit this directly: five parallel agents in five separate git worktrees, each on a separate feature branch, each doing clean work — and the result was branch confusion, cross-contaminated commits, and

orphaned refs that had to be recovered by hand. No agent was undisciplined. The coordination model was wrong. Worktree-per-agent looked like isolation; under the hood it was shared git state with no coordination layer above it, and the humans didn't notice until the integrity was already lost. **The fix:** Stop trying to out-discipline the architecture. When a failure looks like "the agents are good but the result is bad," the thing to change is the *shape* of the collaboration, not the prompts to the agents inside it. Read §11 and ask: are tasks actually independent at the level the coordination model assumes? Are boundaries drawn at the right layer — file, feature, branch, worktree, process? Is there a shared state under the isolation that nobody is managing? The rescue protocol's slogan — *you cannot out-discipline a missing rail* — applies here too, but the rail is the *coordination architecture*, not the pipeline. The third case study's next release pivoted from worktree-per-agent to team-in-one-worktree with file-boundary ownership, and shipped cleanly. Same agents, same discipline, different architecture, different outcome. *Pin: when the agents are good and the result is bad, the architecture is wrong.*

The Verify-What-You-Shipped scar is the canonical version (full story in Part V). The deploy script reported success, every check green; the live site served the old version because an orphan container was still holding the port. "I started the container" had become a stand-in for "reachable from the internet" — a confident lie told by the script, repeated by the agent, believed by us. The fix was one line: `curl` the real health URL and check the body for the new git SHA. *Pin: prose without paste is hallucination. Make the paste mandatory.*

> **Field note — from a real project:** *Verify what you shipped, not what you built.* A green deploy script hid a running-but-unreachable container. Every status claim was technically true; none of them described what a user would see. The fix was a curl against the real public URL, comparing the response body to the SHA we'd just built. **Field note — from a real project:** *The Doorbell That Never Rang.* Both halves of an admin audit-logging integration shipped confidently green, with the wire between them never built and zero rows in the audit table to show for it. Unit tests on each half do not prove the integration exists.

§10 The rescue protocol

You've read §9 and found your failure mode. Good — now you know what's broken. This chapter is the next seven days. It's a checklist, not an essay, because a team in crisis doesn't need more prose. Three horizons: the next hour, the next day, the next week. Work them in order. Don't skip ahead.

In the next hour — stop the bleeding

- **Pick one small, scoped task.** Not the thing that's on fire — something adjacent and contained. Something you can verify end-to-end in five minutes. The goal is not to ship it; the goal is to complete one clean loop so you remember what "working" feels like.
- **Run the full loop on that task.** Brainstorm, even if it's ninety seconds. Plan, even if it's one page. TDD if code is involved. Verify with evidence. Commit. Every step, in order, no shortcuts — you are re-calibrating the muscle, not optimizing it.
- **Stop using the agent for anything you can't verify in 5 minutes.** If you can't check it, you can't ship it right now. Verification is the trust currency; accept IOUs and you'll be babysitting again by tomorrow morning. See §7.
- **Audit your memory files. Delete anything you can't justify.** Open them. Read each entry out loud. If you can't say "this is still true and the agent needs it," cut it. Stale memory is poisoning the well — every contradicted fact teaches the agent that your notes are suggestions. See §5.

In the next day — stabilize

- **Run the brainstorming skill before every task for one full day.** Every single one. The trivial ones too. You will feel it slow you down; it will also catch two scope drifts you would not have caught, and at least one of them would have cost you an afternoon.
- **Write one feedback memory from last week's corrections.** Open your recent chat logs. Find the correction you made most often — the one you're tired of typing. That's the memory. One is

enough. See §5.

- **Run an honest retro on the last week.** Not a changelog. What hurt, what surprised you, what you'd do differently. A paragraph is fine. The point is to name the pattern out loud so you stop walking into it. See §13.
- **Read §7 Verification out loud.** Literally out loud, the whole section. This sounds silly and it works — the failure mode that got you here is almost always evidence discipline, and reading the rules aloud is how they stop being wallpaper and start being rules.

In the next week — rebuild trust

- **Promote two repeated corrections into real skills.** Not another CLAUDE.md paragraph — an actual skill with a trigger and a checklist. If you've typed the same correction three times this month, that's a skill the harness should be loading for you. See §6.
- **Add one hook that enforces a non-negotiable you've been asking the agent to "remember."** The correct number of reminders is zero; the correct number of hooks is one. Pick the rule you are most tired of repeating, and put it somewhere the harness cannot forget it. See §2.
- **Reconcile CLAUDE.md against your memory files.** CLAUDE.md is the things that don't change — architecture decisions, invariants, the shape of the project. Anything that drifts week-to-week belongs in memory or in the trash. Move stale facts out; let CLAUDE.md get smaller.
- **Ship one thing end-to-end using the full loop.** Not a refactor, not a cleanup — a real change that a user would notice, with a real verification that produces real output. Paste the evidence into the commit. Let the feeling of *that worked* compound into the next one.

A worked example: Trust Your Local Tests

Three releases in a row, the team typed some version of "trust your local tests" into a fresh conversation and watched the correction evaporate by the next session. Local tests passed; CI failed. The backend suite ran SQLite on laptops and Postgres in CI, and every round of "works for me" was eroding faith in the suite itself. By the end of that third release, nobody fully believed a green local run meant anything — which meant nobody believed a red one either, which is worse.

The rescue was one cycle of the full loop applied to the parity gap itself, not the symptoms. Brainstorm: *what if local and CI ran the same database?* Plan: rip SQLite out, make the laptop suite use Postgres via Docker. TDD on the harness changes. Verify: same test, same bytes, same result in both environments. Commit, ship, done. The rule it implemented was §7's quiet clause — verification only counts in an environment that matches the one you're shipping to. Once that rail existed, the corrections stopped being necessary. You cannot out-discipline a missing rail. Build the rail.

> **Field note — from a real project:** *Trust Your Local Tests, the rescue that stuck.* The fix wasn't more discipline; it was structural. Rip the SQLite branch out of the test bootstrap, point local and CI at the same Postgres path, stop needing the phrase. Rails that make the old mistake impossible outlast any amount of "please remember to."

Part IV — Leveling Up

§11 Parallel agents & worktrees

Parallelism pays off exactly once: when the tasks are genuinely independent and the overhead of coordination is less than the time the fan-out saves. Everything else — racing on shared state, fan-out as a way to avoid thinking about dependencies, worktrees that quietly accumulate forgotten work — costs more than it earns. This chapter is about knowing the difference before you commit to the shape.

Rule: Fan out only when tasks are independent.

Why: Parallel agents on dependent tasks produce merge conflicts, races, and subtle interleavings you'll spend longer debugging than the parallelism ever saved you. Two agents that both need to edit `services/timer_service.py` will produce a conflict. Two agents that need each other's types but don't wait for them will make different assumptions and ship two incompatible halves of a feature. **How to apply:** If task B needs task A's output, they are sequential. If they touch different files with no shared state, they are parallel. The test is honest: can you describe the handoff between B and A in one sentence? If yes, they are sequential even if the files don't overlap. "B uses the interface A defines" is a handoff. You can make that interface explicit — agree on it in the plan, stub it, and run in parallel — but you have to do that work first, and it counts as a dependency.

The heuristic that holds across every release is grouping by file overlap, not by issue priority. Two high-priority issues that both touch the same test file are not a parallel pair — they are a serialization point. The Parallel Play release is the textbook example of the clean case: one agent writing Python/pytest, one writing TypeScript/vitest, different test runners, different directories, zero shared state. Both produced working tests on first run. The conflict that didn't happen was not an accident — it was a property of the file map. A later release ran 14 subagents across 29 files and landed zero conflicts, for the same reason: the plan mapped file ownership before any agent started work. The file map is the contract. Write it first. *Pin: group by file overlap, not by issue priority.*

> **Field note — from a real project:** *The Parallel Play.* Two agents, two languages, different test runners, zero conflicts. One writing Python and pytest, the other writing TypeScript and vitest, different directories, different build tools, no shared state. The backend agent finished in two minutes; the frontend took three. Both working on first run. The absence of conflict wasn't luck — it was what the file map guaranteed the moment the split was drawn.

> **Field note — from the second case study:** *The Interface That Never Needed a Sixth Method.* A five-method adapter interface — name, start, stop, port, protocol identifier — was designed in the first release and absorbed more than seventy adapters across nine releases without ever changing. TCP, UDP, TLS, HTTP, IPC, raw sockets, binary protocols, text protocols, XML streams, binary-encoded formats, sidecar-based adapters, build-tag-gated platform-specific adapters. Five methods. Still five methods. The retro named the deeper property directly: *"the interface isn't just a software abstraction — it's an organizational one. It turns a team coordination problem into a compilation step."* When four agents are working in parallel against the same interface contract, conflicts are impossible by construction — not by discipline, not by code review, not by merge ceremony, but by the interface refusing to permit them. The agents can build in total isolation and merge without conflict because the only thing each adapter can *do* is satisfy those five methods. Interface-first design is a force multiplier on parallel agents in a way no amount of planning ceremony can match. *Pin: the interface you design before the parallel work begins is the coordination protocol the parallel work runs on. Design it like it matters, because it does.*

> **Field note — from the second case study:** *The Wave Pattern.* By the third release, the project had converged on a rhythm: when you're parallelizing agents on N adapters, do them in waves, not all at once. Trivial adapters first — the ones that are fifty lines of stdlib and fit in a single read. Moderates next — the ones with state machines or transport security. Complex last — the ones with hand-rolled protocol state machines and multi-phase handshakes. The retro named why: *"the trivial wave validates that the build system, config registration, and test harness are all working before you commit agents to 150-line state machines. It's the software equivalent of checking the parachute before jumping."* The wave pattern scaled cleanly to seven parallel agents per release without coordination overhead, because the first wave shook out every infrastructure surprise before the expensive work started. The pattern generalizes: when parallelizing work, schedule the cheapest items first as a load test for the harness. The expensive work runs on a harness you've already debugged.

Rule: Worktrees are workspaces, not stashes — and they are not a coordination strategy.

Why: A worktree is an isolated place to do work. It is not a place to park something "for later." Worktrees rot. Uncommitted experiments accumulate in long-lived worktrees the same way food accumulates on a desk — slowly, invisibly, until what's there is unrecoverable. The cost is not the disk space; it is the work itself: an afternoon of exploration with no commit, no branch, no artifact — just a worktree that got stale and got deleted. **And a second failure mode the original rule missed:** worktrees look like they isolate parallel agents, but the git state underneath them is *shared*. Worktrees isolate the *filesystem*; they do not isolate *branches, refs, HEAD, or merge state*. Five agents in five worktrees on five feature branches are all operating against the same underlying git database, and without an explicit coordination layer above that, they will step on each other's branches, cross-contaminate commits, and produce orphaned refs that have to be recovered by hand. **How to apply:** Every worktree should have a definite end — merged, deleted, or explicitly reopened — within days. If you can't immediately name what the worktree is for, it is time to close it. Clean exit means committed (even as a draft branch), merged, or explicitly discarded — not suspended. "I'll come back to this" is how worktrees die. **For parallel-agent work specifically:** do not reach for worktree-per-agent as the isolation mechanism. Use file-boundary parallelism inside a single worktree, with a team or dispatch layer that assigns non-overlapping files to each agent. The third case study learned this the hard way (see *The v6 Worktree Experiment* below) and pivoted in the next release to a team-in-one-worktree approach with file-level ownership — same agents, same discipline, dramatically different outcome. *Pin: isolate context with worktrees; isolate work with file boundaries. Never rely on worktrees to isolate git state across agents.*

> **Field note — from the third case study:** *The v6 Worktree Experiment.* Five parallel agents, five git worktrees, five feature branches, one codebase. Each agent's individual work was clean. The integration was a disaster: agents committed to wrong branches, commits cross-contaminated between features, and orphaned refs had to be recovered by manually extracting files from abandoned commits and feature branches onto develop. The retro's one-line extract: *"parallel worktrees work best when agents are completely independent; shared git state (branch switching, merges) creates coordination problems."* No agent was undisciplined. The coordination model had a hidden shared-state dimension the setup didn't anticipate. The recovery was hours; the lesson was one release long; the next version did it differently.

> **Field note — from the third case study:** *The v7 Team Pivot.* Same project, next release, same kind of parallel work. Instead of worktree-per-agent with separate branches, the team used a team-spawning mechanism with agents in the *same* worktree, on the *same* branch, with explicit file-boundary assignments before any work started. Three agents ran simultaneously on three separate features — touching completely different file sets (different hooks, different components, different pages) — and merged cleanly with zero conflicts. Two more agents ran in a follow-up phase on additional features with explicit file-boundary instructions to avoid conflicts on shared files like `App.tsx` and `types.ts`. The retro's explicit lesson: *"parallel agents work well when boundaries are drawn at the feature level (different files) rather than the branch level (different git state)."* Same project. Same agents. Same discipline. The architecture changed and the outcome changed. This is what the §9 partnership-architecture failure mode looks like when you fix the architecture instead of the agents.

The right mental model is that a worktree is a sprint, not a shelf. You open it with a task in mind, you work the task, you close it. A worktree that outlives its task is drift in physical form — and unlike a branch, there's no PR to force a reconciliation. If the work is real, commit it. If it's experimental, commit it to a draft branch. If it's done, delete it. The worktree lifecycle should be boring. *Pin: if you can't remember what it's for, close it.*

> **Field note — from a real project:** *The Vitest Cache Incident.* Four parallel agents wrote theme variants across several pages. Each one's output was clean. Thirty-eight unrelated tests started hanging because the concurrent writes had corrupted vitest's module-graph cache — an emergent failure that belonged to no individual agent. Clearing the cache fixed everything. The worktrees had done their jobs; the interaction between them was the invisible one.

Rule: Designate merge points explicitly; update them last.

Why: The hardest part of parallel agent work is not the parallel code — it's the few shared files every agent has to touch: the config registry, the main entrypoint, the integration test, the dependency manifest. If every agent edits those in parallel, you get conflicts by construction. If one agent edits them and the others wait, you've serialized the parallelism. The clean move is to declare those files as *merge points* — shared touchpoints that *nobody* edits during the parallel phase — and update them *after* all the parallel work has landed. The parallel phase becomes conflict-free by construction; the merge phase becomes a single mechanical pass against a known list of files. **How to apply:** Before fan-out, identify every file any agent would plausibly need to touch to integrate its work. That list is your merge-point set. Split the plan into two phases: phase one is the parallel work that touches *only* each agent's own files; phase two is a single-threaded pass that updates each merge-point file with every agent's integration. The file map becomes a contract with two columns: *owned by an agent* and *merge point.* No file can be in both. The second case study ran this pattern cleanly across every release with parallel agents: *"the only shared touchpoints — the config registry, the main entrypoint, the integration test — are updated after all adapters land, eliminating contention."* Six agents, seven agents, twelve agents — zero merge conflicts on core logic across every release that used this pattern. *Pin: parallel work is a two-phase plan. Phase one is the fan-out; phase two is the merge-point pass. Mixing them is where conflicts come from.*

> **Field note — from the second case study:** *Six agents, zero conflicts, by construction.* v0.2.0 had four agents building adapters plus two agents on CI pipeline work — six agents, one codebase, zero merge conflicts on core logic. The reason wasn't discipline; it was that the plan's file map had assigned every file to exactly one writer before any agent started work. The merge points (config registration, main wiring, integration test) were updated in a single-threaded pass after all four adapter agents reported back. The conflict that didn't happen was not a matter of luck or care — it was *impossible* for the structure to produce one.

Rule: Subagents protect your context window; they don't hide work from you.

Why: The point of a subagent is to do a large-process task — search 200 files, read 25 retros, trace a call graph across 14 modules — and return a small output: a decision, a summary, a short report. A subagent that writes 5,000 lines you don't read is a liability, not a feature. You've outsourced the work and the accountability at the same time, and when something breaks in those 5,000 lines the debugging session starts from scratch. **How to apply:** Dispatch when the *output* you need is small but the *process* is large. Research, analysis, searching, reading — good subagent work. Code generation is not exempt from your review just because an agent wrote it. If the subagent's job is to write code, you still have to read the code. The context-window savings from using a subagent do not transfer to the accountability for the code it produces. *Qualifier:* at high parallelism (four, seven, twelve agents at once) the literal "read every line" posture stops being tractable — and in some domains it becomes unnecessary. The second case study's seven-agents-per-release rhythm did not reach every line with a human eye; it reached every line with automated gates. CI running build+test+vet+race on every push, integration tests spinning up every adapter, a coverage matrix test pinning feature coverage — those gates did the reading humans couldn't. The rule is still right about *accountability*: it's yours either way. But "reading" can be mechanized into automated gates when the domain is mechanical enough to support them. This depends entirely on the pipeline being boring (§2). When the gates are trustworthy, you can dispatch broadly and audit narrowly. When they aren't, parallel agent work outruns your ability to supervise it and the whole pattern collapses. See the DevOps playbook's *"fail loudly at known boundaries"* — that rule is what makes this qualifier safe.

Velocity note: at the third case study's tempo — nine major versions, hundreds of tests, PWA, cloud deploy, accessibility pass, and a TanStack Query migration, all in three days — the qualifier stops being an exception and becomes the mode of operation. No human reviewed every line at that cadence; no human could. The humans were doing taste calls, interaction design, milestone scoping, and retro writing, while the agents elaborated against a pipeline that was boring enough to be trusted and a test suite dense enough to catch mechanical regressions. The relationship between the rule and its qualifier inverts at high velocity: automated-gates-as-reading becomes the default, and human line-by-line review becomes a targeted exception reserved for craft-sensitive code — interaction design, data migrations, auth boundaries, anything where taste

or safety is load-bearing. This is the clearest demonstration in the playbook of the three-layer thesis working end-to-end at speed: pipeline catching mechanics, agents handling elaboration, humans spending attention on craft. The velocity isn't the point; the velocity is the *signal* that the three layers are all doing their own jobs.

The Vitest Cache Incident is also this story, viewed from the other side. Four parallel agents wrote theme variants across several pages. The code was correct. The tests were correct. What nobody was tracking was the aggregate effect of four agents making simultaneous writes to a shared `node_modules` directory — a state interaction that existed outside any single agent's view and outside the review that followed. The fix was mechanical (`rm -rf node_modules/.vitest`), but finding it required diagnosing an effect that emerged from the parallel structure itself, not from any individual agent's output. What you dispatch, you own — including the parts that emerge from interactions between dispatched tasks. *Pin: dispatch to compress process, not to avoid reading the result.*

> **Field note — from a real project:** *The Subagent Orchestra.* Fourteen subagents across two parallel tracks, 29 files touched, zero merge conflicts. The plan's file map — written before any agent started — assigned every file to exactly one dispatch. Conflicts were impossible by construction, not by discipline.

§12 Plan quality

A plan is a contract you write with future-you. Future-you will be context-deprived, possibly stressed, definitely without the reasoning you had during planning. The plan is either detailed enough to carry that reasoning forward, or it isn't — and if it isn't, future-you will fill the gaps with whatever seems plausible in the moment. That is where features drift and bugs are born.

Rule: A plan with placeholders is a wish list.

Why: "TODO: handle errors later" is the engineer outsourcing the hard part to future-them. Future-them will be confused, stressed, and without context. The hard part has to be done *at plan time*, not at implementation time. **How to apply:** Every step contains the actual content. No "TBD," no "similar to above," no "implement later," no "add appropriate error handling." If you can't say what the error handling is, you're not ready to plan it yet — go back to brainstorming.

The shape of a placeholder looks reasonable. "Handle edge cases" sounds responsible. "Wire up the error path" sounds thorough. Neither tells you what the edge cases are, what the error path does, or what the downstream behavior should be when it fires. When the agent hits that step, it fills the gap with something plausible. Not wrong, exactly — but not the thing you had in mind, either, and by the time you notice you've built two layers on top of it. The tell is a plan that you can read and nod along to but couldn't execute yourself. If you couldn't hand it to a new team member and walk away, it isn't specific enough. Go back. Name the thing.

The self-updating preflight check from the blue-green deploy release is the inverse of this problem stated positively: instead of hardcoding a list of required environment variables in the script — effectively "TODO: remember to update this list when we add a new var" embedded in deploy infrastructure — the script learned to read required vars from the compose file itself. A single grep against the compose YAML. The list is always current because the plan was complete enough to ask "what is the actual source of truth for required vars?" instead of proxying it with a placeholder that would go stale. A plan that asks the right question before deferring never has to ask it again.

> **Field note — from a real project:** *The Preflight That Would Have Saved Us a Week.* An earlier deploy shipped with a missing environment variable because the script's required-vars checklist was hardcoded and three vars behind the compose file. The fix was self-updating validation: the preflight grep'd the compose file for variable references and checked each one was set. The list was always current because nobody had to remember to update it. Self-updating validation beats a manually-maintained checklist, every time. *Pin: if you can't fill in the step right now, you can't implement it right now.*

Rule: Each step is one action, two to five minutes long.

Why: Bigger steps hide complexity and break the TDD rhythm. "Implement the feature" is a chapter, not a step. Small steps also mean small blame radius — when a step fails you know exactly where. **How to apply:** "Write the failing test" is a step. "Run the test and verify it fails" is a step. "Write the minimal implementation" is a step. If a step would take longer than five minutes to complete, split it. *Qualifier:* the five-minute rule is a *proxy* for "small blame radius." The real constraint is that when a step fails, you should know exactly where without needing to decompose what the agent did. When the architecture hands you small blame radius for free — a one-adapter-per-directory layout, a tight interface contract, a self-contained package with its own test file — a single step can be larger than five minutes and still satisfy the rule. The second case study's unit of parallel work was "implement the next adapter" — sometimes thirty minutes, sometimes a couple of hours — and this was fine because a failure in any single adapter was trivially localized: one directory, one package, one test file, one agent. The step was big; the blame radius was still small. Check the proxy against the goal — if architecture is already keeping blame radius small, steps can relax. If it isn't, hold to five minutes.

The failure mode is a step that sounds atomic but isn't. "Add date range filtering to the admin list" — sounds like one thing. Actually it's: write the query parameter model, add the backend filter clause, write the service method, add the frontend filter state, wire the UI controls, add the E2E test, and handle the edge case where the end date is before the start date. Seven steps collapsed into one. When the agent "does" that step, it makes all seven decisions at once and you get to review seven decisions at once — which means you'll miss the one that's wrong. The Month That Wasn't Thirty Days hotfix is a clean example: the planning step was "write tests for the 30-day date filter." The step that would have caught it was "write a test that generates dates 30 days apart" and then a *separate* step "verify that 30-day offset doesn't cross a month boundary on CI." The first step sounds sufficient; the second is where the flakiness lived. Step granularity is how you surface the complexity before the agent buries it.

> **Field note — from a real project:** *Hotfix 1: The Month That Wasn't Thirty Days.* A test that used a 30-day date offset worked fine most months and failed on calendar days when "30 days ago" crossed a month boundary. The plan step "write tests for the 30-day date filter" was too coarse to name the constraint; a separate step — "verify that the 30-day offset doesn't cross a month boundary on CI" — would have caught it at plan time instead of from a user report. *Pin: if a step takes longer than five minutes, split it.*

Rule: The plan must cover the spec.

Why: A plan that drifts from the spec produces a feature that drifts from the requirement. Every section of the spec without a corresponding task is not a gap in the plan — it is a gap in the feature. **How to apply:** After writing the plan, walk every spec section and point at the task that implements it. If a section has no task, you have a hole — not a feature, a hole.

The Four Scoping Gaps release is the canonical case. The spec was clear: the release-automation tool should open Release PRs on the develop branch and trigger deploy when a Release is published. The plan was one step — "add `target-branch` to the config" — that sounded like it covered the spec. It did not. Walking the spec against the plan would have surfaced four separate gaps (cataloged in Part V), each plausible in isolation, none visible unless you walked the spec step by step and asked "which task covers this?" The test for spec coverage is pointed and mechanical: take each requirement sentence and name the task that delivers it. If you can't name one, write one.

The Trust Your Local Tests positive case is worth holding alongside it. That spec broke the work into five phases and was explicit about which items were already solved by earlier phases — one issue was marked "no code changes needed, resolved by an earlier phase"; another had been fixed in a prior release. One whole phase required zero commits. The spec knew this before the engineers did, because they'd written it down. Spec coverage in the forward direction lets you close issues before the agent ever touches the keyboard.

> **Field note — from a real project:** *Four Scoping Gaps, spec-walk edition.* Four scoping gaps in one release-automation plan, each of which looked reasonable in isolation and none of which a

spec walk would have allowed. The plan had one step; the spec had four requirements; the math was right there on the page if anyone had written both columns next to each other. *Pin: walk every spec section and name the task that implements it.*

Qualifier: spec coverage includes *temporal* ordering across systems. The "plan covers the spec" rule, as written, catches missing tasks. It does not by itself catch *tasks executed in the wrong order across systems whose dependencies cross the boundary between them.* When a plan touches more than one system — code + infrastructure, code + database migrations, code + secrets vault, code + DNS, code + third-party config — the order of operations is part of the spec, even when the spec doesn't say so out loud. A plan that says "add startup-time validation that requires `IP_SALT` to exist" must include the step "add `IP_SALT` to the secrets vault" *and* must execute that step *before* the validation deploys. Otherwise the validation deploys, the process refuses to boot, and the deploy bricks itself the instant it lands. The fix is one line of plan; the cost of skipping it is an outage. Treat cross-system ordering as a first-class spec coverage check: for every task, ask "what must already be true in some other system before this task can run?" — and add the prerequisite as its own earlier task.

> **Field note — from a real project:** *The Guard That Came Before the Secret.* A release added a "fail fast" startup check: if `IP_SALT` wasn't set in the environment, the API would refuse to boot rather than fall back to a hardcoded default that would silently weaken the IP-hashing scheme. The PR was clean. Tests passed. Code review approved it. The deploy script ran. The new container started. The startup check fired. The container exited. The deploy script retried. The container exited. Two retries later the deploy script gave up and rolled back to the previous version. Root cause: the plan added the *guard* but never added the step "set `IP_SALT` in the production secrets vault" — which a teammate had assumed was already there because the variable name had appeared in earlier code. It hadn't been. The fix was thirty seconds (add the secret, redeploy). The lesson is the rule above: **a plan that introduces a precondition must also introduce the step that establishes the precondition's prerequisite, in the right order.** Walking the spec is not enough; you also have to walk the *dependencies between systems* and verify the order of operations across them. *Pin: when a task assumes another system is in a particular state, the assumption is part of the spec — and the step that gets the system into that state is part of the plan.*

Rule: Descope explicitly. Name what's out, and name why.

Why: A plan that only names what's *in* is a plan that quietly hopes everyone agrees on what's *out*. They won't. The items you silently omit will come back as surprise asks during implementation, as scope creep during review, or as "wait, I thought we were doing that" at the retro. Explicit descopes close those loops at plan time, when they're cheap, instead of at implementation time, when they're a conversation you didn't plan to have. **How to apply:** In every plan, write a dedicated "Descoped" section. List every item the plan considered and decided *not* to do. Give each one a named reason — "hardware-dependent, untestable in CI," "dependency too large for this release," "spec evolves faster than we can keep up," "low value for the complexity cost." A descope with a reason is a negotiable contract. A descope without a reason is an argument waiting to happen. The second case study's final release is the clean example: v1.0.0 cut four items from the plan, each with a one-line reason, and the retro treated those cuts as *part of the release's value*, not as a failure to ship everything. Descoping with reasons is how a plan stays honest about its own boundaries.

> **Field note — from the second case study:** *Descoping the Final Four.* The v1.0.0 release cut four items that had been on the roadmap, each with a one-line reason: one required physical hardware and was untestable in CI; one brought in a massive dependency with complex signaling; one depended on an enormous external codebase for a spec that evolves rapidly; one added significant complexity with no proportional value. Each descope was named with its reason in the release plan *and* in the retro. The release still shipped well above its original target, and the four cuts were part of the story, not a shortfall. A plan that can explain what it chose not to do is a plan that has actually been thought about. *Pin: every plan has a descoped section. Every descoped item has a reason.*

> **Field note — from the second case study:** *The Config Test That Was Always One Release Behind.* A single test in the project's config package asserted the exact number of registered adapters — a hardcoded integer that had to be bumped every single release because the test pinned a literal instead of deriving it. One retro caught it mid-flight: *"TestDefaultConfig failed with the wrong expected count. The ghost of the previous release, hard-coded in an assertion, politely informing us that we had changed the thing we explicitly set out to change."* The deeper lesson is that tests which pin *literal facts about the codebase* (counts, filenames, fixed lists) are decoupled from the facts they claim to pin, and every release has to remember to update them manually. A test that derives its assertion from the code is self-updating; a test that hardcodes it is a scheduled reminder disguised as an assertion. *Pin: if a test has to be updated every time the code changes, the test is asserting the wrong thing.*

§13 The retro habit

Every loop ends with a retro. Not a changelog — a retro. The loop is brainstorm → plan → TDD → verify → commit → retro, and the last step is the one that turns a cycle into cumulative learning. Without it, each release is isolated. With it, each release teaches the next one something the last one had to discover the hard way.

This is the second load-bearing chapter of the playbook, and it is paired with §7. Verification catches the failure; retros convert it into a rule. Skip verification and the failure ships. Skip the retro and the failure re-ships, forever. Every scar in Part V exists because one of those two disciplines was skipped. Every rule in the rest of this playbook exists because one of those two disciplines produced it. If you remember nothing else from this document, remember that pairing: **verify the work to catch the bug; retro the cycle to kill the class.** Two disciplines, one practice, and the reason the playbook has any rules at all.

Four projects have independently evolved this discipline. The first case study's twenty-four narrative retrospectives, the second case study's nine themed release retros, the third case study's nine-version retro-plus-workflow-document practice, and the fourth case study's audit-driven retros all converged on the same shape without any of them copying from the others. When four differently-shaped projects independently reach for the same tool, the tool is load-bearing, not incidental. The retro is not a ceremony; it is the mechanism by which agent-directed work becomes cumulatively smarter instead of cyclically forgetful. This chapter is load-bearing because the practice is load-bearing because the discipline is load-bearing. Everything else in the playbook is downstream of this.

Rule: Write retros in voice, not in bullet points.

Why: A dry changelog is forgotten in a week. A story is remembered in a year. The retrospective's job is to be *rediscoverable* — and humans rediscover stories, not checklists. Name the bugs by their nicknames. Describe what it felt like. Admit what surprised you. The difference between "Release 5.11 CI/CD Cleanup Summary" and "The One Where Five Small Fixes Grew Teeth" is not aesthetic — it is the difference between a document nobody opens and one somebody searches for by instinct two years from now. If your retro reads like a status report, it will be treated like one: filed, ignored, and eventually auto-archived by a sprint tool.

How to apply: Write like a person, not a project manager. If you spent three deploys fixing a bug that turned out not to exist, that's a story worth telling exactly that way. If a Slack integration blew up because Slack's format is named after Markdown the way ketchup is named after tomatoes, say that. The engineer who finds your retro eighteen months from now while Ctrl-F'ing for "Slack notifications broken" will thank you for the specificity. Titles are half the job: "The One Where We Solved The Wrong Bug For Three Rounds" is something someone will click. A version number followed by a feature list is not.

> **Field note — from a real project:** *Titles earn their clicks.* The first case study has 24 retros, from the founding sprint through the most recent pipeline polish. Each one has a title that tells you what it's about. One is "The One Where Five Small Fixes Grew Teeth." One is "The Preflight That Would Have Saved Us a Week." Another is "Hotfix 1: The Month That Wasn't

Thirty Days." The retros are the proof of concept for this rule — and they're the source material for many of the field notes in this playbook. *Pin: "The One Where We Solved The Wrong Bug For Three Rounds" is a title someone will click in two years. "Release 5.11 CI/CD Cleanup Summary" is not.*

Rule: Retros are how the loop learns.

Why: Without a retro, every cycle re-discovers the same mistakes. The lesson learned in retro N becomes the rule applied in loop N+1 — but only if retro N gets written. Skipped retros compound. The first skip is free; by the third, you've stopped seeing the pattern you're walking into because you never named it the first time. The instinct to skip is strongest on small releases. Those are exactly the ones that teach the most granular lessons — the kind that aren't worth a postmortem but are worth a paragraph.

How to apply: A retro is mandatory at the end of every release, even small ones. It doesn't have to be long. It has to be honest. An honest paragraph beats a dishonest page — and a missing retro is the most dishonest thing you can write, because its absence implies nothing went wrong, and something always went wrong. The Health Check That Wasn't root-cause analysis is three paragraphs and four sentences of "what we learned." That's enough. The Month That Wasn't Thirty Days retro was written in 20 minutes about a flaky test that only failed on certain calendar days. Six months later it was cited in a planning document. Paragraphs don't evaporate; meetings do.

> **Field note — from a real project:** *The One Where Five Small Fixes Grew Teeth.* The Health Check That Wasn't cost three rounds of correct fixes to a non-bug because nobody stopped to ask what `/health` actually returned. The retro named it. The pattern — verify the verifier — is now a rule in §7. That transfer happened because the retro was written. *Pin: skipped retros compound.*

Rule: Write a retro at the end of every themed cycle, and right after every surprise.

Why: "End of every release" is a useful default only when your work has releases. When it doesn't — or when "release" is a blurry calendar thing rather than a shippable thematic cycle — the default collapses and retros start getting skipped for the wrong reasons. The rule that works across contexts is: write a retro whenever a cycle closes *or* whenever something surprises you. Cycles give you scheduled reflection. Surprises give you opportunistic reflection. Together they catch both the slow lessons (the accumulation of small rough edges over a themed slice of work) and the fast ones (the unexpected bug, the near-miss, the "huh, that wasn't supposed to happen"). A cycle without a retro leaks its slow lessons. A surprise without a retro leaks its fast ones. **How to apply:** Retro at three triggers.

- **At the end of any themed cycle with a shippable boundary.** The §4 nested-cycles rule is the driver: task-level, release-level, and project-level cycles each deserve a retro at their own scale. A task-level retro can be three sentences and a lesson. A release-level retro is the full story. A project-arc retro is the meta-story of all the release retros.
- **At the end of any unit of work that took more than a day**, even if no one calls it a release. If you spent a day or more on something, something happened that is worth a paragraph.
- **Immediately after any surprise** — a major incident, a near-miss, a debugging session that went sideways, a deploy that rolled back, a test that failed in a way that made you say "huh." Surprise is the signal that the cycle already produced a lesson worth extracting *now*, not at the next scheduled boundary. Catch it while the context is hot.

Do not write retros on calendar intervals divorced from the work ("weekly retros regardless of whether anything happened"). A retro with nothing to say is worse than no retro, because it teaches the team that retros are performance, not learning. Retros follow cycles and surprises, not dates.

> **Field note — from the second case study:** *Nine retros, one per release, none skipped.* The second case study wrote a retro at the end of every single release — nine of them across the major arc, plus the follow-on point releases. Every one had a theme, a story, nicknamed bugs, and a "what we learned" section. The discipline wasn't "retros on Friday"; it was "the cycle

closes, the retro gets written, then the next cycle can begin." The retros were the latch between cycles. Without the latch, cycle nine doesn't know what cycle three learned. *Pin: the retro is the latch between cycles. No retro, no latch, no cumulative learning.*

Rule: A retro has an anatomy. Use it as a scaffold, not a template.

Why: "Write in voice, name what broke, end with one extractable sentence" is a *posture*, not a *structure*. A reader who hasn't seen a good retro has nothing to start from. The posture rules (voice, honesty, extractable lesson) produce great retros *once you know the shape they live in*. The shape is the thing that makes a retro skimmable by a cold reader, re-readable by a future engineer looking for something specific, and parseable by an agent extracting lessons. Without the shape, retros drift into freeform memoir that is readable exactly once. **How to apply:** The scaffold, in five parts. Scale each part to what the cycle actually produced — not every retro needs every part, but the shape is what makes a longer retro legible.

1. **The mission.** One paragraph on what you were trying to do, why it mattered, and what "done" was going to look like. A cold reader eighteen months from now needs this to orient. Skip it and the rest of the retro is context-free.
2. **What happened — the execution narrative.** The story of the cycle. Nicknamed bugs. Surprises. The near-misses that didn't make the commit log. The parts that were harder than expected and the parts that were easier. Written in voice (§13 rule 1), not in bullet points. This is where the humans live.
3. **The numbers.** Dry metrics for the people who came for them: counts, times, sizes, test results, dependency deltas, lines added and removed. Not the whole build output — the handful of numbers that characterize the cycle. This section makes the retro skimmable by a reader with a specific question and no time for the story.
4. **What we learned — the extractable lessons.** Stated explicitly, not left for a reader to infer from the story. One sentence per lesson, with the lesson first and the context second. This is the section the agent and future-you will Ctrl-F for. It's also the section that feeds the next cycle's plan (§4 nested cycles). If a lesson isn't in this section, it isn't going to reach the next loop.
5. **What's next.** The handoff to the next cycle. What this retro is telling the next loop to do differently, to watch for, or to try. A one-line *"what's next"* is enough; the point is to make the lesson actionable on the very next cycle.

Not every retro needs all five. A one-day hotfix might be three sentences and a lesson. A release retro probably wants all five. The anatomy is a scaffold for when the retro is larger than a paragraph — it's what keeps a long retro from becoming an unstructured memoir that nobody re-reads. The second case study's nine release retros all follow roughly this shape: mission, waves (their execution-narrative section), numbers, what we learned, what's next. They read like stories *because* they have skeletons underneath.

> **Field note — from the second case study:** *The retro as story with a skeleton.* Every release retro opens with a one-sentence mission, walks through waves of execution with nicknamed adapters and discovered gotchas, lists the raw numbers in a "The Numbers" section, extracts lessons in a "What We Learned" section, and closes with "What's Next." The shape is unmistakable from retro to retro, but the voice is different every time — each has its own rhythm, its own jokes, its own arc. The skeleton is what lets the voice happen: when the structure is handled, attention goes to the prose. *Pin: the anatomy is the scaffold that frees the voice.*

Rule: A retro has three audiences: you next month, your team, and the agent next time.

Why: The retro you write today is read by future-you when you're trying to remember what you did, by teammates who weren't there, and by an agent that will be asked to read this retro as context for a future task. All three need different things — future-you needs the full story, teammates need the context they missed, and the agent needs a clean extractable lesson — but all three can get what they need from the same document if you include the lesson explicitly, not just as something implied by the narrative.

How to apply: Every retro ends with "what we'd do differently" — a sentence, not a section. That sentence is the thing the agent extracts. That's the sentence you'll Ctrl-F for next month. It doesn't have to be long.

The Four Scoping Gaps retro's extract is: "Walk every spec section and name the task that implements it before starting the work." Trust Your Local Tests': "If local and CI aren't running the same test suite, fix the suite, not the gap." Those sentences are now rules in §12 and §7, respectively — they got there because they were written down explicitly, not left inside the story for a reader to infer. The story is for the humans. The explicit lesson is for everyone, including the agent that hasn't read the story yet.

> **Field note — from a real project:** *Four Scoping Gaps, three-audience edition.* The retro named four distinct failures rather than smearing them into one undifferentiated "it didn't work." An agent reading that retro before the next similar task would have known, one sentence at a time: `target-branch` is an action input, not a config key; the release-automation tool plus the default workflow token does not trigger downstream workflows; `release:` is not a conventional commit type; and a missing anchor release sends the tool scanning the whole history. All three lessons were in the retro as explicit extractable sentences, not buried in paragraphs of narrative. *Pin: every retro ends with one sentence the agent can extract.*

The first case study has twenty-four retros behind it, written across two years, each one a story the field notes in this playbook have been borrowing from. Part V collects the ones worth reading in full — alongside entries from the other three case studies — and here they are, ready when you are.

Part V — Field Notes from the Case Studies

Every rule in this playbook has a bruise under it. The entries below are where the bruises came from — compact re-tellings of the bugs that taught us the rules, each one naming the chapter it grounds. Read them in any order; each is self-contained. The headlines are whimsical on purpose, because a bug with a nickname is a bug you remember.

The stories are grouped by the *kind* of problem they illustrate, so a reader scanning for "I'm having a verification problem" or "my parallel agents are stepping on each other" can jump to the right neighborhood first. Within a group, they're in no particular order.

Verification & feedback loops

Stories where the test was green, the status was "ok," and the thing was broken. The bruises here all trace back to verifying the wrong surface or trusting a signal that was answering a different question than the one you were asking.

The Health Check That Wasn't (v5.11.0)

What happened: Three deploys in a row, `git_sha` came back `unknown` in the health response after every release. We rebuilt the Docker image pipeline four different ways — shell export of `GIT_REV`, compose env file, compose override, `--build-arg` — and each fix deployed cleanly and "didn't work." **What we thought was happening:** The build-arg plumbing was dropping `GIT_REV` somewhere between the YAML and the container, and we just had to find where. **What was actually happening:** The verification curl was hitting `/health`, which returns `{"status":"ok"}` and has never had a `git_sha` field. The full health JSON lives at `/api/v1/health`. The producer was never broken. We spent three deploys fixing a hole that wasn't there because the verifier was looking through the wrong window. **The lesson:** §7 — health checks must check health, and verification curls must hit the endpoint that reports it. *(See DevOps Playbook Gotcha #8.)*

Verify What You Shipped, Not What You Built (v5.9.0)

What happened: The deploy script reported success. The new container had started. Every status check was green and every log line said "ok." The live site was still serving the old version. **What we thought**

was happening: A browser cache, a CDN delay, something between the user and the server — not the deploy itself. **What was actually happening:** An orphan container from the previous compose definition was still holding the port. The new container was running and healthy and completely unreachable from the internet, because nginx had never flipped to it. "I started the container" had become a stand-in for "the container is serving traffic." The fix was one line of verification: one `curl` against the public health URL, comparing the response body to the SHA we just built. **The lesson:** §7 — evidence before assertions, against the URL the user will actually hit.

Trust Your Local Tests (v5.7.0)

What happened: For three releases running, the backend test suite passed on every laptop and failed intermittently in CI. Nobody trusted "local green" anymore — every PR got a manual "but did it pass in CI?" even after it had passed in CI. **What we thought was happening:** Flaky tests. A Redis timing issue. Something environmental we'd catch eventually. **What was actually happening:** Local used SQLite via a `is_testing` branch in `database.py`; CI used Postgres. Every release, the gap introduced a new dialect-specific failure, and every release we patched the symptom. The rescue was a single cycle of the full loop applied to the gap itself — rip the SQLite branch out, make `conftest.py` truncate a shared Postgres engine between tests, force every environment through the same path. **The lesson:** §7 — verify in the environment that matters, and §10 — you cannot out-discipline a missing rail.

The Cache That Worked Locally And Lied In CI (v5.4.0)

What happened: We refactored the cache layer so Pydantic models flowed through Redis, shipped the PR with a green local suite, and watched an image-generation pipeline explode in CI the moment the deploy ran. **What we thought was happening:** A Redis connection issue, or maybe a dependency version mismatch in the CI image. **What was actually happening:** Someone had written `json.dumps(model, default=str)` as the serializer. `default=str` does not dictify a Pydantic model — it calls `str()` on it and produces the model's *repr*, a string that looks like Python source. Local was green because local used an in-memory cache with no serialization path at all; the model went in as an object and came out as an object. CI had actual Redis, which needed JSON, which exposed the bug. **The lesson:** §7 — if your local doesn't run what CI runs, your local green is a lie.

The Doorbell That Never Rang (v4.2.0)

What happened: Audit logging shipped. Backend tests green, frontend tests green, a beautiful admin page that rendered an audit table. The table had zero rows in it. Every admin action completed successfully and left no trace. **What we thought was happening:** A query filter bug, or a timezone off-by-one making events land outside the default window. **What was actually happening:** Nothing in between the two halves emitted an audit event. Backend exposed the read endpoint. Frontend rendered the list. No service call anywhere in the admin routes actually fired the emit. Both halves were tested in isolation; nobody had wired the doorbell to the button. A thirty-second click in a real browser would have caught it. **The lesson:** §7 — "tests pass" is not "feature works." Open the thing and use it.

Scope, plans, and surprises

Stories where the plan looked obvious, the scope felt small, and the thing that bit was the work nobody had named yet. These are the bruises that come from skipping the brainstorm, leaving a placeholder in a plan, or trusting a fan-out to hide coordination problems it never could.

Four Scoping Gaps in One Release (v5.11.0)

What happened: A one-line release-automation config fix was supposed to make Release PRs open against the develop branch and trigger the deploy workflow when a Release was published. The fix shipped. Nothing worked. Then the second fix shipped. Nothing worked. **What we thought was happening:** A typo in `release-please-config.json` — add `target-branch`, move on. **What was actually happening:** Four

separate scoping errors, each plausible in isolation. `target-branch` is an action input, not a config key — the JSON accepted it silently. There was no anchor Release, so release-please scanned to the start of the repo. The merge commit used `release:` as its type, which is not a conventional-commit type and was silently skipped. And release-please plus the default `GITHUB_TOKEN` does not trigger downstream `release: types: [published]` workflows. **The lesson:** §12 — walk every spec section and name the task that implements it.

The Vitest Cache Incident (v5.0.0)

What happened: Four parallel agents wrote theme variants across several pages. The code was clean, the tests were clean, the dispatch was textbook. Then thirty-eight unrelated tests started hanging with vitest timeouts, every one of them in files nobody had touched. **What we thought was happening:** An agent had subtly broken a shared util or introduced a render loop somewhere in a component mount path. **What was actually happening:** Four agents had been writing to the same `node_modules` directory at once, and vitest's module-graph cache under `node_modules/.vitest` had corrupted itself during the concurrent writes. No individual agent's output was wrong. The failure was an emergent property of the parallel structure. `rm -rf node_modules/.vitest` fixed all thirty-eight tests. **The lesson:** §11 — subagents protect your context window, they don't hide interactions between dispatched tasks from you.

The Four-Fix WCAG Contrast Cycle (v5.5.0–v5.6.0)

What happened: Across two releases, four separate WCAG contrast violations slipped into the UI — each one a utility-class color swap picked by eye that looked fine in the design tool, failed the axe-core suite on the next run, and got fixed with a one-class swap. **What we thought was happening:** We thought we were calibrated to the contrast rules by now. After the second one, we thought we were calibrated *now*. After the third, we stopped guessing. **What was actually happening:** Nothing was broken — the automated rail was catching exactly what it was designed to catch. The humans were the slow learners. The right move was to stop pre-judging contrast by eye and let the axe-core evidence do the approving. **The lesson:** §8 — when the blast radius is small and the test is automated, confirm by evidence, not by ceremony.

Hand-Rolling the Legacy State Machine — Right Call, Wrong Call (second case study, v0.3.0)

What happened: The second case study was implementing a batch of early-era protocols. One of them is the pathological one — a decades-old protocol that uses *two connections* for what every other protocol accomplishes with one, with a secondary data channel that requires hand-negotiated host/port encoding in a format nobody else uses. The team considered importing a library and decided instead to hand-roll the entire state machine from the standard library. ~150 lines. Pure stdlib. Listener, data channel, directory listing, file transfer. All of it. **What we thought was happening:** This is a judgment call about dependency weight versus implementation cost. The project's ethos was zero-dependency-if-possible; the protocol was in scope; the hand-roll was tractable; we'd save the dep. **What was actually happening:** The retro named the tension directly: *"Right because it kept the dependency count low and forced us to truly understand the data-channel negotiation. Wrong because we now truly understand the data-channel negotiation, and that knowledge cannot be unlearned."* The hand-roll worked. It shipped. The test suite ran clean. And the team now carries around the protocol's internal mechanics as permanent mental residue — the bespoke host-port encoding, the ephemeral listener, the two-connection dance, the state transitions that make sense only if you were in the room when the spec was written. Every unit of knowledge about a legacy protocol you gain is a unit of attention you cannot reclaim for anything else. The dependency you didn't import was free; the attention you spent internalizing the protocol is not. **The lesson:** A counter-weight to "always reach for a library" *and* to "always hand-roll for purity." The real question isn't library-vs-hand-roll — it's whether the knowledge you'll acquire during the hand-roll is worth the craft attention it will permanently occupy. Sometimes yes (the team's zero-dependency posture became a source of genuine craft pride across nine releases). Sometimes no (the legacy protocol's mechanics are nobody's art). The judgment call connects back to §1's three-layer thesis: a hand-roll is a decision to spend *craft* attention on *mechanics*, and that trade is only worth it when the mechanics are themselves part of the craft. Here they aren't. The zero-dependency

posture as a whole is. The retro was honest about the tension instead of pretending the hand-roll was unambiguously correct, which is the only reason the lesson survives.

Tooling & environment traps

Stories where the code was fine and the tool was lying. Bash buffering its own script, a token that couldn't wake the next workflow, a Slack dialect cosplaying as Markdown — the bruises here all come from trusting a tool to behave like its documentation said it did.

The Actions Token That Won't Wake the Next Workflow (v5.10.0)

What happened: We moved release creation to release-please and the deploy pipeline went silent. Merging the Release PR created the GitHub Release exactly as intended; the `deploy.yml` workflow, wired to `release: types: [published]`, never fired. **What we thought was happening:** A YAML trigger typo, or maybe a branch-protection rule eating the event. **What was actually happening:** A documented GitHub Actions behavior: PRs and Releases created by a workflow using the default `GITHUB_TOKEN` do not trigger downstream workflows. The "fix" was technically working — a Release was created — but the next stage was built on a chain that the token could not carry across. A PAT or GitHub App token was required for the handoff. **The lesson:** §7 — verify in the environment that matters; a green release event doesn't prove the downstream workflow saw it.

Bash Buffers Scripts In Memory (v4.2.0)

What happened: The footer on the live site read `rev unknown` after deploy. The footer had displayed the git hash since v4.0.0. It worked in dev. It worked in CI. It did not work in production. We pushed fix after fix to `deploy.sh` and watched the old buggy version run to completion anyway. **What we thought was happening:** Git wasn't pulling, or the runner had a stale clone, or the `GIT_REV` export was being eaten somewhere in the compose plumbing. **What was actually happening:** Bash reads a script into a memory buffer when it starts executing. `git pull` updated the file on disk — the new bytes were right there — but the running shell kept executing its in-memory copy of the old file. Any edits after the `git pull` line never took effect on the run that pulled them. The fix is one line: after the pull, `exec "$0" "$@"` to re-read the script from disk. **The lesson:** §9 — when the obvious fix doesn't stick, stop editing and dump the raw state. *(See DevOps Playbook Gotcha #9 for the `exec "$0" "$@"` self-reexec pattern.)*

Slack mrkdwn Is Not Markdown (v5.10.0)

What happened: CI notifications started going out to the team channel with raw `**asterisks**` and `[link text](urls)` rendered as literal characters instead of formatting. The messages looked like a drunk bot. **What we thought was happening:** A webhook payload escaping bug — something double-encoding the Markdown before Slack got it. **What was actually happening:** Slack's mrkdwn is not Markdown. Bold is `*single asterisks*`, not `**double**`. Links are `<url|text>`, not `[text](url)`. The webhook was delivering exactly what we'd written; we had written the wrong dialect. The fix was a full conversion to Block Kit, where the formatting contract is explicit instead of cosplaying as a familiar one. **The lesson:** §7 — verify in the environment that matters, including the one that renders your message. *(See DevOps Playbook Gotcha #6 and Phase 6 for the Block Kit migration.)*

Trust & blast radius

Stories about authorization, denial paths, and the moments when "yes" in one direction quietly failed to mean "yes" in every other direction. The bruises here come from forgetting that the interesting rules live in what gets refused, not in what gets allowed.

The Admin Who Couldn't Fire Herself (v4.3.0)

What happened: RBAC landed for the admin panel. The interesting bugs weren't in the "yes" paths — those were one-liners. They were in the "no" paths: self-role-change, self-deactivation, self-deletion, privilege

escalation via update. **What we thought was happening:** RBAC is a matrix of allows; fill it in and you're done. **What was actually happening:** Trust in one direction ("you are an admin") does not generalize in every direction ("therefore you may fire yourself"). Half a dozen explicit refusals had to be written on purpose, one by one, because each was a line the matrix didn't draw by default. The last-superadmin protection came later still, in v5.6, because we missed it the first time too. **The lesson:** §8 — authorization is scoped, not blanket. Every yes is for one action.

Coda — The three layers, read back through the field notes

Every field note in Part V is a moment when the three-layer division of labor from the thesis was violated. Read them that way and the whole gallery becomes one story told twelve ways.

- **The agent was asked to do the craftsperson's job.** *The lookup form called "clutter" and removed* (referenced in §1): a design-pass agent made a taste call about what was load-bearing for a use case it had no context for. That's craft work, and craft work is the human's. The agent was doing elaboration the human should have done itself. The fix wasn't a better prompt — it was the human keeping taste in the human lane.
- **The pipeline was trusted to validate the thing it wasn't validating.** *The Health Check That Wasn't*, *Verify What You Shipped*, *The Docker Port Mappings That Weren't*, *The bufconn Gap*: a pipeline reported green while the thing the user actually hits was broken. In every case, the mechanics layer was doing *a* job, just not *the* job — and the human had stopped supervising because the green check looked authoritative. The fix wasn't more discipline; it was making the pipeline check what actually mattered.
- **The human was doing the laborer's job.** *Trust Your Local Tests*: three releases of typing the same correction into fresh conversations, because the parity gap had turned the human into a permanent manual rail. The fix was to build the rail in code so the human could stop being it. *The Four Scoping Gaps*: a fifteen-minute brainstorm — craft work — was skipped because the problem looked mechanical, and the human ended up doing four passes of mechanical debugging instead of one pass of craft thinking.
- **One layer tried to compensate for a missing layer.** *The Onion*, *Three Deploys to Green*: weeks of cascading failure because the CI and deploy layers were doing work the pipeline should have been doing by itself. The human and agent were fighting the mechanics layer, not working inside it.

The thesis isn't decoration. It's the diagnostic. When a cycle feels wrong, ask which layer is doing the wrong job, and put it back in its lane. Every rule in this playbook is, in the end, a way to perform that diagnosis quickly and return each layer to itself — so the human can get back to the work only a human can do.

The audit is craft work the human owns

One craft discipline deserves naming explicitly because it has no analog in the agent or the pipeline layer: **the systematic audit.** Not a feature pass, not a bug-fix sprint, not a refactor — an audit. Looking at your own work with fresh eyes, on purpose, with no goal except to find what's wrong. The fourth case study has two clean exemplars worth pinning: a **security audit pass** in one release that surfaced fifteen issues a normal review cycle had walked past for a year (lost-update races, TOCTOU on entity creation, missing UUID validation, regex injection vectors, materialized views that were never being refreshed) — and a **visual audit pass** in a later release that walked every page of the app in both light and dark themes on both desktop and mobile, with no agenda, and produced six UX fixes nothing else had surfaced (inconsistent page header sizes, button overload, featured content styled identically to errors, a confidence bar taking three rows for one piece of information).

Neither pass was a feature release. Neither was driven by a user complaint or a bug report or a failing test. Both came from the human deciding *"I'm going to look at our work, systematically, until I see what we missed."* And both produced more value than any feature work that release could have shipped — because the things they found were invisible by every other instrument the team had.

This is craft work the agent cannot substitute for, and the pipeline cannot catch. The agent doesn't audit; it executes. The pipeline doesn't audit; it gates. The audit is a human act of *attention* — slow, deliberate, ungoaled looking — and the only person on the team who can do it is the one who knows what the work is *for*. Auditing is exactly the kind of attention the three-layer thesis is trying to free up. The pipeline is boring so the agent can elaborate; the agent elaborates so the human can audit; the human audits so the work gets *good*.

Schedule audits on purpose. Pick a surface — security, accessibility, performance, visual consistency, error messaging, onboarding flow, a single component family — and walk it end to end with no goal except "find what's wrong." The first one will feel like overkill. The findings will silence that. *Pin: every release that ships only what the bug reports asked for is a release that left work on the table. Audit on purpose.*

We can create art and beauty with a computer. The three-layer discipline is how you make that line true in practice, every day, on a real project with real deadlines. The agent elaborates. The pipeline validates. You create — and audit, and notice, and decide what's worth making. The playbook is the apparatus. The art is the point.

Appendix A — If you are an agent reading this

You are an agent, and a human engineer has loaded this playbook as context because they want you to work the way it describes. This appendix is your index: every rule from chapters §1 through §13, stripped to imperatives, each with a one-sentence reason and a link back to the full section. Skim the rules here; jump to the section when a rule needs context you don't have. One overriding instruction: if anything in this appendix conflicts with the project's `CLAUDE.md`, with a skill the user tells you to run, or with a direct instruction from the user, the user wins — always. The rest of the file is for the human reading over your shoulder; you can skim it, but the rules live here.

§1 Mental models · full section

- **Brief the agent like a smart colleague who just walked into the room.** — Every prompt is a cold start; a self-contained briefing prevents invented answers.
- **Delegate the task, not the understanding.** — Synthesis is the engineer's job; handing the agent a riddle is where work falls apart.
- **The agent is good at breadth and consistency. It is bad at judgment under ambiguity.** — Use it for search, refactor, fan-out; make trade-off calls yourself.

§2 The workspace · full section

- **Your pipeline is a precondition, not a feature.** — Every discipline in this playbook assumes CI catches mechanical failures quickly and deploys are automated; a flaky pipeline inverts the loop and drags the human into the mechanics layer. You are done when the pipeline is boring. See the companion DevOps playbook.
- **CLAUDE.md is for facts that don't change. Memory is for facts that do.** — Stable facts in a stable layer stay correct; moving facts in memory can be corrected next turn.
- **Skills are for procedures. CLAUDE.md is for facts.** — Skills get executed on purpose; notes get absorbed as ambient noise and ignored under pressure.
- **Hooks make automation non-negotiable.** — Anything the agent is asked to "remember to do" will fail silently at least once; hooks take the choice out of the loop.
- **`settings.json` configures the harness. CLAUDE.md configures the agent. They are different layers.** — "Can't" is enforced by the tool; "should" is enforced by politeness.

§3 The first conversation · full section

- **Bootstrap with the smallest context that contains the answer.** — Token budget is attention budget; three load-bearing files beat eighty skimmed ones.
- **Correct drift in message 2, not message 50.** — Early corrections are nearly free; late ones mean rewriting ten turns built on top of the drift.
- **If the agent doesn't know something, tell it. Don't let it guess.** — Hallucination is most likely when the agent is confidently filling a gap.

§4 The loop · full section

- **Run the whole loop. Skip a step out loud or pay for it silently.** — Each step (brainstorm, plan, TDD, verify, commit, retro) catches a different class of failure.
- **Brainstorm before planning. Plan before code.** — Out of order, the bounds are hallucinated; a plan that feels obvious hasn't been pressure-tested. The brainstorm can amortize across cycles when the upfront thinking was rigorous and the work is repetitive.
- **Frequent commits are not optional.** — Bisecting agent bugs only works if commits are small enough to bisect against. Precondition: a pipeline fast enough to support the cadence.
- **Retros feed the next loop.** — Without them, the loop has no memory and each release starts from zero.
- **Cycles nest. Each level needs a theme, a boundary, and all three phases.** — Task-level, release-level, and project-level loops all take the same shape; each needs a nameable theme, a shippable boundary, and an explicit upfront-plan / execute / validate structure. Cycles without themes get named by date and forgotten.

§5 Memory hygiene · full section

- **Memory has four types. Use the right one or it rots.** — User facts, feedback, state, and pointers have different staleness profiles; mixing them produces a junk drawer.
- **Stale memory is worse than no memory.** — No memory makes the agent ask; stale memory makes it act confidently on last quarter's facts.
- **Save the why, not just the rule.** — A rule without its reason becomes unfollowable dogma the moment the world moves on.
- **Don't memorize what the code already says.** — Anything one `grep` away will drift out from under a memory note the moment a file is renamed.

§6 Skills as institutional knowledge · full section

- **Skills are procedures with discipline. CLAUDE.md notes are facts you hope get followed.** — If a thing must happen reliably, it needs a host other than the agent's attention.
- **Rigid skills exist for a reason. Don't adapt the discipline away.** — Rigid skills feel like overkill in exactly the moments they're needed most.
- **Write a skill after the third correction.** — The third correction is the moment "I keep saying this" becomes "the system should enforce this."

§7 Verification before completion · full section

- **"Tests pass" is not "feature works." Verify the feature.** — Suites can be 100% green while the feature is broken because no test exercised the wire between parts. Infrastructure migrations specifically need tests on the *interaction* pipeline, not just the *data* pipeline.
- **Evidence before assertions. Always.** — Never claim success without producing the verification command and its actual output; prose without paste is hallucination. And evidence of the *wrong claim* is still theater — name the claim the evidence actually supports before accepting a green check.
- **Verify in the environment that matters.** — If local doesn't run what CI runs, local green is a lie.

- **Health checks must check health, not "is the process running."** — A check that can't detect a downed dependency is a status check cosplaying as a health check.

§8 Trust boundaries · full section

- **Match the action to its blast radius. Confirm before crossing the line.** — Blast radius is set by recovery time, not by command length.
- **Authorization is scoped, not blanket.** — Every yes is for one action; it is not a standing permission slip for the next risky thing.
- **When the agent hits an obstacle, it must investigate, not delete.** — Unexpected files and orphan containers usually represent in-progress work; if you can't show where you looked, you didn't look.

§9 Failure modes & recovery · full section

- **Agent went off the rails:** stop, open a fresh conversation, and write a self-contained briefing (§3) — steering a context-starved conversation only gives new ways to be wrong.
- **Agent produced slop:** add explicit YAGNI constraints and promote them into memory or a skill (§5, §6) — without a rail, every prompt drifts toward gold-plating.
- **Agent can't be trusted with anything important:** install evidence-before-assertions discipline hard (§7) — you can't out-discipline a missing rail; build the rail.
- **Agent loops on the same wrong fix:** stop touching code, dump raw state, diagnose the real failure (§7) — when the second fix fails the same way as the first, the root cause is misdiagnosed.
- **Agent confidently lies about state:** reject any success claim not accompanied by verification command plus output (§7) — this is the failure that looks like progress and rots underneath.
- **Partnership architecture is wrong:** the agents are good but the result is bad — change the shape of the collaboration, not the prompts (§11). If file boundaries aren't clear, work overlaps; if worktrees are being used as a coordination strategy, git state leaks between agents. Fix the topology, not the discipline.

§10 The rescue protocol · full section

- **In the next hour:** pick one small scoped task, run the full loop on it, stop using the agent for anything you can't verify in 5 minutes, and audit memory files with a bias toward deletion — re-calibrate the muscle before optimizing it.
- **In the next day:** run the brainstorming skill before every task for a full day, write one feedback memory from last week's corrections, run an honest retro, and read §7 out loud — stabilize by reinstalling the disciplines that got skipped.
- **In the next week:** promote two repeated corrections into real skills, add one hook that enforces a non-negotiable, reconcile CLAUDE.md against memory, and ship one thing end-to-end with pasted evidence — move pain from prompts into the harness.

§11 Parallel agents & worktrees · full section

- **Fan out only when tasks are independent.** — Parallel agents on dependent tasks produce merge conflicts and subtle interleavings; group by file overlap, not issue priority. An interface-first design is the strongest coordination protocol — when every agent can only produce code that conforms to a narrow contract, conflicts become impossible by construction.
- **Designate merge points explicitly; update them last.** — Shared touchpoints (config, main, integration tests) should be owned by a single post-parallel pass, not edited by the parallel agents. Phase one is fan-out; phase two is the merge-point pass.
- **Worktrees are workspaces, not stashes — and they are not a coordination strategy.** — Worktrees rot when they outlive their task; if you can't name what it's for, close it. And they do not isolate git state across parallel agents — use file-boundary parallelism inside a single worktree for multi-agent work, not worktree-per-agent.

- **Subagents protect your context window; they don't hide work from you.** — Dispatch when the output is small but the process is large; you still have to read the code. At high parallelism, ”reading” can be mechanized into automated gates when the domain supports it — but accountability for the produced code is always yours. At very high velocity the qualifier becomes the default mode.

§12 Plan quality · full section

- **A plan with placeholders is a wish list.** — If you can't fill in the step right now, you can't implement it right now.
- **Each step is one action, two to five minutes long.** — Small steps surface complexity before the agent buries it. The time budget is a proxy for small blame radius; when architecture provides small blame radius for free, steps can be larger.
- **The plan must cover the spec.** — Walk every spec section and name the task that implements it; a section with no task is a hole in the feature.
- **Descope explicitly. Name what's out, and name why.** — A plan that only names what's in is a plan that silently hopes everyone agrees on what's out. Every plan gets a ”descoped” section with named reasons.

§13 The retro habit · full section

- **Write retros in voice, not in bullet points.** — A dry changelog is forgotten in a week; a story with a nickname is rediscoverable in two years by instinct.
- **Retros are how the loop learns.** — A retro is mandatory at the end of every release, even small ones; an honest paragraph beats a dishonest page. §13 is the second load-bearing chapter of the playbook, paired with §7: verification catches the failure; retros convert it into a rule.
- **Write a retro at the end of every themed cycle, and right after every surprise.** — Scheduled retros catch slow lessons; surprise retros catch fast ones. Don't retro on calendar intervals divorced from the work.
- **A retro has an anatomy. Use it as a scaffold, not a template.** — Five parts: mission, execution narrative, numbers, what we learned, what's next. Scale each to what the cycle produced; the scaffold is what frees the voice.
- **A retro has three audiences: you next month, your team, and the agent next time.** — End every retro with one extractable ”what we'd do differently” sentence — the story is for humans, the explicit lesson is for everyone including the agent that hasn't read the story yet.

Appendix B — Glossary

Terms with local meaning — reach for this when a word is doing more work than it seems to.

- **Blast radius** — the scope of consequences if an action goes wrong; used to decide whether to confirm with the user before running a destructive command. See §8.
- **Blue-green deploy** — a deployment pattern that runs the new version (”green”) alongside the old (”blue”), health-checks it, flips traffic, and keeps the old version warm for rollback. *(See DevOps Playbook Phase 4.)*
- **Conventional commits** — a commit-message convention (`feat:`, `fix:`, `chore: …`) that machine tools can parse to decide version bumps and changelog sections. *(See DevOps Playbook Phase 5.)*
- **Health endpoint** — an HTTP endpoint that reports the service's own dependency status and build identity (git SHA, version), distinct from a lightweight status check used by load balancers. *(See DevOps Playbook Phase 7.1 and Gotcha #8.)*
- **release-please** — a GitHub Action that reads conventional commits, opens Release PRs, and publishes GitHub Releases automatically. *(See DevOps Playbook Phase 5 and Gotchas #3–#5.)*
- **Brainstorm** — the first step of the loop; a skill invocation that turns a vague idea into a written spec. See §4.

- **CLAUDE.md** — the project-instructions file the harness loads automatically as system context at the start of every conversation. Claude Code calls it CLAUDE.md; other harnesses use different filenames (`.cursorrules`, `AGENTS.md`, `GEMINI.md`) for the same role. See §2.
- **Field note** — a short inline vignette that grounds a rule in a real event from the case studies; appears in the human layer of every two-layer chapter. See How to read this playbook.
- **Flexible skill** — a skill whose steps are guidelines to adapt to context, as opposed to a rigid skill that must be followed exactly. See §6.
- **Hook** — a shell command the harness executes automatically on a named event (e.g., `post-tool-use`, `pre-commit`) without waiting for an explicit agent invocation. See §2.
- **Memory** — one of four typed persistent notes (user, feedback, project, reference) that an agent writes and reads across conversations to preserve context beyond a single session. See §5.
- **Plan** — a written, step-by-step implementation guide derived from a spec that defines how to build something before any code is touched. See §12.
- **Rigid skill** — a skill whose steps must be followed exactly in order, with no discretionary adaptation. See §6.
- **settings.json** — the harness-level configuration file that controls which tools are available, what permissions are granted, and which hooks are registered. See §2.
- **Skill** — an invokable, named procedure with a defined discipline; the harness construct, not the colloquial sense of "ability." See §6.
- **Spec** — a written design contract that describes what to build, captured before planning begins; a plan covers how to satisfy a spec. See §12.
- **Subagent** — a fresh child agent dispatched with a curated context window, used to protect the parent's context or to run isolated work in parallel. See §11.
- **The loop** — the canonical development cycle: brainstorm → plan → TDD → verify → commit → retro; every feature passes through every step. See §4.
- **Two-layer format** — the playbook's chapter convention: an imperative rule at the top (the agent layer) followed by narrative explanation and a field note (the human layer). See How to read this playbook.

Appendix C — About the case studies

Every field note in this playbook — every nicknamed bug, every three-deploy chase, every "we thought it was X, it was actually Y" — comes from one of four real projects. The first is a full-stack production web application built by a small team over roughly two years, heavy on agent collaboration from early days, shipped through something like twenty-four public releases from its founding sprint to its most recent pipeline-polish cycle. It's live. It has users. It has the kinds of scars a project gets when you ship things on purpose, read your own retros, and write the next one anyway. This appendix intentionally anonymizes product identity across all four case studies — the point of the playbook is the practice, not the products — so what's described below is the *shape* of each effort: team size, velocity, release cadence, coordination pattern, and what each one added to the rules.

The retrospective practice is the part that matters for this playbook. Every release — feature, patch, infrastructure cycle, emergency hotfix — got a narrative retro at the end. Not a changelog and not a feature list: an actual story, written in voice, with the bugs named by the nicknames they earned while people were fixing them. *The Doorbell That Never Rang. The Health Check That Wasn't. The Month That Wasn't Thirty Days. The Vitest Cache Incident.* The discipline of writing those retros is what generated the raw material this playbook is curated from. Part V's field-note gallery is the short list — the entries that transferred cleanest to a rule — but every other story that shows up inline, every "from a real project" vignette, traces back to one of those retros.

They appear in this playbook as inline vignettes rather than as links for one reason: the reader is not assumed to have access to the source. That is the whole point of this portable version. Every story this playbook actually relies on is told in full, right where it's cited. You do not need to click anywhere. If the vignette feels thin, the rule above it is still carrying the weight; if the vignette feels thick, you have what you need

without leaving the page.

The same practice, applied to any project with any tech stack, would generate the same *kind* of material. Your bugs will not be our bugs. Your deploy script will fail in ways ours never did. Your agent will invent helpers your team will never use. Those become your field notes — and they will, at the time you are writing them, feel exactly as load-bearing as ours did to us.

The specific project is not the point. The *practice* is the point. Your project's field notes are going to look different and be just as load-bearing.

Second case study — a parallel-agent protocol server

Partway through curating this playbook, we pressure-tested it against a second project with a very different shape: a single-binary server whose work was implementing a large catalog of external network-protocol specifications as plug-in adapters behind a narrow shared interface. Over nine releases and roughly six months, it grew from three adapters to more than seventy, spanning TCP, UDP, TLS, HTTP, IPC, binary wire protocols, text protocols, XML streams, binary-encoded formats, and build-tag-gated platform-specific primitives. The project is a single Go binary with two external Go dependencies and two sidecar Python dependencies, approximately 15,000 lines of code, and a five-method interface that never changed across any release. It was built almost entirely by parallel agents — four, six, seven, sometimes twelve at once — with humans holding the planning, retros, and judgment calls.

The shape is completely unlike the first case study. No production database. No user-visible UI. No deploy drama (it ships as a container image and a docker-compose file). No quarterly release calendar. The work is rhythmic, mechanical, and heavily parallelizable by construction. If the first case study is *"one team shipping a web app to real users for two years,"* the second is *"can a small group of humans direct a fleet of agents to implement dozens of protocol specs, and keep the results coherent?"* Different question, different scars.

What held without modification. Verification (§7), the loop (§4), retros (§13), memory hygiene (§5), trust boundaries (§8), and plan quality (§12) all landed in the second case study the same way they landed in the first. The core rules are not artifacts of one project's shape — they're the disciplines that make agent-directed work legible at all.

What the second case study amplified. Parallel agents (§11) gained three new rules' worth of evidence: the interface-as-coordination-protocol insight, the wave pattern, and the explicit merge-point discipline. Every one of these was implicit in the first case study and load-bearing in the second. A five-method adapter interface absorbed more than seventy adapters across every protocol family in networking history without ever changing — the strongest evidence for interface-first design this playbook contains.

What the second case study stress-tested. Three rules needed qualifiers rather than rewrites: agent judgment (§1) is stronger when trade-offs are mechanical, delegating understanding (§1) is safer when the understanding lives in authoritative external specs, and the five-minute-step rule (§12) is a proxy for small blame radius that can relax when the architecture provides it for free. The first case study couldn't have surfaced any of these qualifiers because its work lives on the other end of each spectrum — social trade-offs, head-based understanding, judgment-heavy steps. Two projects, two positions on each axis, one honest set of rules.

What the second case study added. Two entirely new rules that the first case study had no way to generate: the nested-cycles rule in §4 (the second case study's nine themed releases are the clean exemplar) and explicit descoping in §12 (its v1.0.0 "Descoped Four" is the clean exemplar). Both rules were already implicit in the first case study's practice; the second case study's different shape is what forced them out into the open as explicit rules.

The practice that generated the second case study's material is the same one that generated the first's: a narrative retro at the end of every release, written in voice, with nicknamed adapters and named lessons. **The Wave Pattern. The Interface That Never Needed a Sixth Method. The Docker Port Mappings That Weren't. Hand-Rolling the Legacy State Machine.** The second case study's retros

sit alongside the first's as the raw material for the playbook's second pass — and reading them against each other is how every field note from the second case study in this document earned its place.

Two projects, very different shapes, same practice, same rules — and this was only the second pressure-test.

Third case study — a high-velocity time-tracking web app

Partway through the second pressure-test, we brought in a third project: a time-tracking web app (React, TypeScript, Supabase, Netlify) built to support retroactive block painting rather than real-time timers. The shape is closer to the first case study than to the second — full-stack web app with a real auth surface, a real database, a real cloud deploy, a real user-facing UI — but the velocity is on a different planet. The third case study shipped **nine major versions in three days**, from scaffold to PWA: local foundation, week view and analytics, goals/streaks/CSV export, cloud deploy, security hardening, accessibility and keyboard shortcuts and undo/redo, TanStack Query migration plus sharing plus PWA, half-hour granularity across ~35 files, and a pure tech-debt release to pay off carried debt. Two hundred unit tests, an E2E suite, axe-core accessibility tests, Lighthouse CI, Sentry, Playwright on chromium and mobile, all running as gates.

The project has its own codified workflow document — `docs/WORKFLOW.md` — that independently describes the same practice this playbook describes: CLAUDE.md as single source of truth, versioned backlog with themed milestones, changelog per release, narrative retrospectives per version, git-flow, dual-deploy with backend-first ordering, quality gates with visual QA, and AI agent coordination via team spawning and file-boundary parallelism. None of this was copied from the first case study's practice; the third case study's team arrived at the same shape from first principles.

What held without modification. Every rule in the playbook that applied. The loop, retros, verification, memory, skills, trust boundaries, plan quality — the third case study ran all of them and benefited from all of them. The three-layer thesis is the most visibly true on this project: at nine-versions-in-three-days, the human cannot be doing laborer's work anywhere in the stack or the tempo collapses. The velocity is the signal that all three layers were doing their own jobs.

What the third case study added. Two contributions that weren't present in the first two projects.

The first is the **v6 → v7 worktree pivot**, which forced a new failure mode into §9 and a new qualifier onto §11's worktrees rule. v6 tried five parallel agents in five separate git worktrees on five separate feature branches and produced branch confusion, cross-contaminated commits, and orphaned refs that had to be recovered by hand. No agent was undisciplined. The coordination architecture had a hidden shared-git-state dimension the setup didn't anticipate. v7 pivoted to a team-in-one-worktree approach with explicit file-boundary ownership, and shipped cleanly. Same agents, same discipline, different architecture, different outcome. This is the clearest demonstration in any of the case studies of a failure that is *not* a discipline problem and *cannot* be fixed by installing more discipline — you have to change the topology of the collaboration itself. The partnership-architecture failure mode in §9 exists because of this release.

The second is the **v7.0.1 and v7.0.2 hotfix pair**, which sharpened §7's "evidence before assertions" rule with a new angle the playbook didn't previously name. Both hotfixes shipped within 24 hours of release despite a full green test suite — because the tests answered a different question than the humans assumed. The TanStack Query migration broke cache-hit paint because no test exercised the cache-hit path; the iPad tap failed because no test exercised a touch gesture on a real device. The unit tests were not lying. They were answering the question *"does the data layer work?"* when the humans were reading the green check as *"does the feature work?"* Those are different claims, and the distance between them is where this class of bug lives. The playbook now names this directly: *the right evidence for the wrong claim is still theater.*

What the third case study stress-tested. One rule stretched rather than broke. §11's subagent qualifier — "automated gates can count as reading when the domain supports it" — was written as an *exception* clause on the main rule. At this project's tempo it becomes the *default* mode, with human line-by-line review reserved for craft-sensitive code (interaction design, data migrations, auth boundaries). The rule is still correct; the relationship between the rule and its qualifier inverts at high velocity. I noted this inline rather than rewriting the rule.

Fourth case study — an audit-driven community-data web app

The fourth pressure-test was a community-data web application with admin moderation: React 18 + Vite + TypeScript + Tailwind on the frontend, edge functions in a managed serverless runtime on the backend, managed Postgres for storage, OAuth for optional user accounts, Vitest for unit tests, Playwright for integration. The shape overlaps the first and third case studies — full-stack web app with a real auth surface, a real database, real users, real moderation workflows — but the distinguishing feature is the explicit *audit cadence* that runs alongside feature work: a scheduled security audit pass in one release produced fifteen findings that a year of normal review cycles had walked past (lost-update races, TOCTOU on entity creation, missing UUID validation, regex injection vectors, materialized views that were never being refreshed), and a later visual audit pass walked every page of the app in both light and dark themes on both desktop and mobile, producing six UX fixes nothing else had surfaced. Neither audit was a feature release; neither was driven by a user complaint. Both came from the human deciding *"I'm going to look at our work, systematically, until I see what we missed"* — and both produced more value than any feature release that cycle could have shipped.

What held without modification. Same as the others — the loop, retros, verification, memory, skills, trust boundaries, plan quality. The fourth case study ran them all.

What the fourth case study added. The **systematic audit as craft work** — a rule now pinned explicitly in the Coda. The first three case studies all had audits in their retros, but only this project ran them as *scheduled* work with their own releases and their own retros, and the results made it impossible to treat auditing as a nice-to-have. Audits are now named in the playbook as a discipline the agent cannot substitute for and the pipeline cannot catch — the one irreducibly human move that the three-layer thesis is trying to free attention for.

It also sharpened the *extract-when-the-pattern-stabilizes-vs-duplicates-drift* discussion in §6 with the `naivePlural()` scar: a small utility function copy-pasted across three edge functions instead of extracted into a shared helper, one copy of which silently drifted by forgetting to handle irregular nouns, producing user-visible "1 mice" / "1 mouses" inconsistencies across code paths. Duplicates are fine when the pattern is still finding its shape; duplicates that have already drifted are a bug waiting to be reported. The fourth case study is what moved that distinction from implicit practice into an explicit clause on the rule.

Four projects, four different shapes, same practice, same rules — plus the small set of sharpenings each new project forced out of the rules that already existed. If the rules hold across four independent projects with different shapes, different languages, different velocities, different team sizes, and different review cadences, the rules are not artifacts of any one project. They are the shape of the practice itself.

Companion — the DevOps playbook

This playbook has a sibling: the **DevOps playbook**, derived from the same first case study and the same retro practice. The two documents are explicitly paired. The DevOps playbook tells you how to make the pipeline boring — health endpoints that report real dependencies and the build SHA, blue-green deploys with automated rollback, lint/test/scan on every push, conventional commits and release-please, manifest-driven production state, Docker parity between dev and prod. Its thesis is one line: *you are done when the pipeline is boring.* This playbook tells you how to work inside a boring pipeline with an agent.

Read together, the two playbooks are the two halves of a single practice. The DevOps half builds the mechanics layer; the engineer-agent half works inside it. The thesis — *we can create art and beauty with a computer* — depends on both. Without the pipeline half, the agent becomes something you babysit and the human becomes the laborer. Without the engineer-agent half, the boring pipeline just runs empty cycles under nobody's direction. Neither playbook is self-sufficient. Both together are.

If the rules in this document feel like they're leaning on a precondition you don't have — fast CI, reliable deploys, parity between environments — go read the companion first. Build the rails. Come back. Everything here will cost less once you do.